

Documentación Completa de Jules

Manual Exhaustivo de Usuario, API, CLI, Integraciones y Changelog

Google Jules Agent Docs
Fuente: <https://jules.google/docs>

Índice General por Módulos

Módulo 1: Guías y Documentación General

- [Getting started](#)
- [Environment setup](#)
- [Running Tasks with Jules](#)
- [Scheduled Tasks](#)
- [Suggested Tasks](#)
- [Reviewing plans & giving feedback](#)
- [Reviewing code changes](#)
- [Managing tasks and repos](#)
- [Repo view](#)
- [Errors and failures](#)
- [Limits and Plans](#)
- [Feedback and support](#)
- [FAQ](#)
- [Contact](#)

Módulo 2: Referencia de la REST API

- [Quickstart](#)
- [API Reference](#)
- [Authentication](#)
- [Sessions](#)
- [Activities](#)
- [Sources](#)
- [Types Reference](#)

Módulo 3: Herramientas CLI de Jules (Jules Tools)

- [Jules Tools Reference](#)
- [Practical Examples & Scripting](#)

Módulo 4: Guías e Integraciones

- [Continuous AI Overview](#)
- [Integrations](#)
- [Render Integration](#)

Módulo 5: Registro de Cambios (Changelog)

- [Changelog](#)
- [Gemini 3.1 Pro is now available in Jules](#)
- [Auto-Fixing CI Failures and configure Jules to commit as you](#)
- [MCP support comes to Jules](#)
- [Gemini 3 Flash is now the base model in Jules](#)
- [Update Scheduled Tasks](#)
- [REST API: Repoless, File Outputs, and Activity Filters](#)
- [Jules now finds performance optimizations](#)
- [Introducing the Planning Critic for Auto-Approved Plans](#)
- [Automated fixes for Render deployments](#)
- [Put routine maintenance on autopilot with Scheduled Tasks](#)
- [Enable suggested tasks to let Jules find issues proactively](#)
- [Start from scratch—instantly](#)
- [Introducing Gemini 3 Pro](#)
- [New Jules Tools CLI Updates: Side-by-Side Diffs, Repo Inference, and More](#)
- [Introducing the Jules API](#)
- [Jules in the command line](#)
- [Use Environment Variables In Jules](#)

- [Jules gains memory!](#)
- [Tell Jules exactly what file to work on using file selector](#)
- [Jules Acts on PR Feedback](#)
- [All Hands on Deck!](#)
- [Image upload](#)
- [Stacked Diff](#)
- [Improved Jules Critic](#)
- [Jules Sample Prompts](#)
- [Render images in the diff viewer](#)
- [Increasing the VM Size to 20GB](#)
- [Export at any time](#)
- [Critic Agent](#)
- [Interactive Plan](#)
- [Jules can surf the web](#)
- [Jules can test web-apps and show you the results](#)
- [Jules is out of beta!](#)
- [Environment snapshots for faster tasks](#)
- [Open A PR directly from Jules](#)
- [Added Bun runtime support](#)
- [Improved task controls and other 🍷 UI delight](#)
- [Jules now listens to GitHub issues](#)
- [Jules Agent Update: Faster, Smarter, More Reliable](#)
- [Modernized base environment and updated toolchains](#)
- [Customization and Efficiency Enhancements](#)
- [A faster, smoother and more reliable Jules](#)
- [Improving Stability](#)
- [Jules is here](#)

Módulo 1: Guías y Documentación General

Getting started

Módulo 1 | Fuente: <https://jules.google/docs/>

Getting started

Jules is an experimental coding agent that helps you fix bugs, add documentation, and build new features. It integrates with GitHub, understands your codebase, and works **autonomously** — so you can move on while it handles the task.

This guide will walk you through setting up Jules and running your first task.

Login

[Section titled "Login"](#)

1. Visit jules.google.com
2. Sign in with your Google account.
3. Accept the privacy notice (one-time).

Connect GitHub

[Section titled "Connect GitHub"](#)

Jules needs access to your repositories in order to work.

1. Click **Connect to GitHub account**.
2. Complete the login flow.
3. Choose *all* or specific repos that you want to connect to Jules.
4. You will be redirected back to Jules. If not, try refreshing the page.

Once connected, you'll see a **repo selector** where you can select the repo you want Jules to work with, and a prompt input box.

Starting your first task

[Section titled "Starting your first task"](#)

Jules runs in a virtual machine where it clones your code, installs dependencies, and modifies files.

1. Pick a repository from the repo selector.
2. Choose the branch you want Jules to work on. The default branch will be selected already. You do not have to modify this unless you want Jules to work on a specific branch.
3. Write a clear, specific prompt. For example, `Add a test for "parseQueryString" function in utils.js`
4. (Optional) Add environment setup scripts.
5. Click **Give me a plan**

Once you submit a task, Jules will generate a plan. You can review and approve it before any code changes are made.

Include AGENTS.md file

[Section titled "Include AGENTS.md file"](#)

Jules now automatically looks for a file named AGENTS.md in the root of your repository. This file can describe the agents or tools in your codebase, such as what they do, how to interact with them, or any input and output conventions. Jules uses this file to better understand your code and generate more relevant plans and completions.

Tip: Keep AGENTS.md up to date. It helps Jules and your teammates work with your repo more effectively.

Enabling notifications

[Section titled "Enabling notifications"](#)

You are free to leave Jules while it is running. To stay informed:

1. When prompted, enable browser notifications.
2. Go to **Settings** —> **notifications** at any time to enable or disable notifications.

You'll be notified when the task completes or needs your input.

What's next?

Section titled "What's next?"

- [Running a task with Jules](#) - Full walkthrough
- [Environment setup](#) - Make Jules smarter about your project
- [Reviewing plans & feedback](#) - how to approve and integrates

Want to dive into real-world use cases? Check out the [Jules Awesome Prompts repo](#).

Environment setup

Módulo 1 | Fuente: <https://jules.google/docs/environment/>

Environment setup

Jules runs each task inside a secure, short-lived virtual machine (VM). This lets it clone your repository, install dependencies, and run tests.

For simple environments, Jules studies your repository to learn how to quickly setup your environment without a provided setup script. Jules will also refer to agents.md or your readme.md file for hints to setup an environment on the fly.

For compe~~x~~x environments, you can provide a **setup script** to be run explicitly that prepare the environment.

What's preinstalled?

[Section titled "What's preinstalled?"](#)

Every Jules VM runs Ubuntu Linux and includes many popular developer tools out of the box:

- Node.js
- Bun
- Python
- Go
- Java
- Rust

The latest versions can be seen here:

Environment check output

```

----- Python -----
✓ python3: Python 3.12.11
✓ python: Python 3.12.11
✓ pip: pip 25.1.1 from /home/jules/.pyenv/versions/3.12.11/lib/python3.12/site-packages/pip (python 3.12)
✓ pipx: 1.4.3
✓ poetry: Poetry (version 2.1.3)
✓ uv: uv 0.7.13
✓ black: black, 25.1.0 (compiled: yes)
✓ mypy: mypy 1.16.1 (compiled: yes)
✓ pytest: pytest 8.4.0
✓ ruff: ruff 0.12.0
✓ pyenv: available
system
3.10.18
3.12.11 (set by /home/jules/.pyenv/version)

----- NodeJS -----
✓ node: v22.16.0 *
  v18.20.8 *
  v20.19.2 *
  -- v22.16.0 *
  system *
✓ nvm: available
✓ npm: 11.4.2
✓ yarn: 1.22.22
✓ onpm: 10.12.1
✓ eslint: v9.29.0
✓ prettier: 3.5.3
✓ chromedriver: ChromeDriver 137.0.7151.70
  (dfa4dc56b2ahb56eb2a14cad006ea5e68c60d5de-refs/branch-heads/7151@{#1875})

----- Java -----
✓ java: openjdk version "21.0.7" 2025-04-15
  OpenJDK Runtime Environment (build 21.0.7+6-Ubuntu-0ubuntu124.04)
  OpenJDK 64-Bit Server VM (build 21.0.7+6-Ubuntu-0ubuntu124.04, mixed mode, sharing)
✓ maven: Apache Maven 3.9.10 (5f519b97e9448438d878815739f519b2eade0a91d)
✓ gradle: Gradle 8.8

----- Go -----
✓ go: go version go1.24.3 linux/amd64

----- Rust -----
✓ rustc: rustc 1.87.0 (17067e9ac 2025-05-09)
✓ cargo: cargo 1.87.0 (99624be96 2025-05-06)

----- C/C++ Compilers -----
✓ clang: Ubuntu clang version 18.1.3 (1ubuntu1)
✓ gcc: gcc (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0
✓ cmake: cmake version 3.28.3
✓ ninja: 1.11.1
✓ conan: Conan version 2.17.0

----- Docker -----
✓ docker: Docker version 28.2.2, build e6534b4
✓ docker: Docker Compose version v2.36.2

----- Other Utilities -----
✓ awk: GNU Awk 5.2.1, API 3.2, PMA Avon 8-g1, (GNU MPFR 4.2.1, GNU MP 6.3.0)
✓ curl: curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2
  (+libidn2/2.3.7) libssh/0.10.6/openssl/zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.7
✓ git: git version 2.49.0
✓ grep: grep (GNU grep) 3.11
✓ gzip: gzip 1.12
✓ jq: jq-1.7
✓ make: GNU Make 4.3
✓ rg: ripgrep 14.1.0
✓ sed: sed (GNU sed) 4.9
✓ tar: tar (GNU tar) 1.35
✓ tmux: tmux 3.4
✓ yq: yq 0.0.0

```

You can check installed versions by adding commands like `node -v` to your setup script and clicking **Run to Validate**.

To view all preinstalled tools, you can use this command in your setup script and click **Run to Validate**.

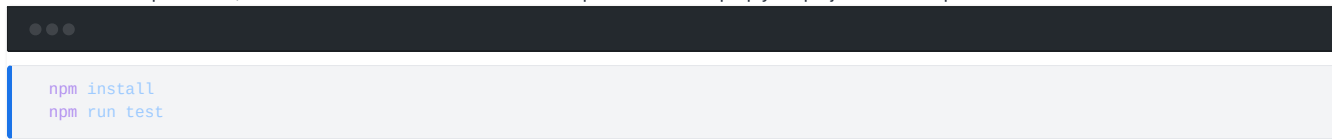
```
set +x; . /opt/environment_summary.sh
```

Add a setup script

[Section titled "Add a setup script"](#)

To help Jules install dependencies and run tests:

1. Click on the repo on the left sidebar (under codebases)
2. Select Configuration at the top
3. In the "Initial Setup" window, enter the commands needed to install dependencies and prep your project For example:

A screenshot of a terminal window with a dark background. It shows two lines of text: 'npm install' and 'npm run test' in a light blue font. The terminal has three small circles in the top left corner, representing window controls.

```
npm install
npm run test
```

Test your setup script

[Section titled "Test your setup script"](#)

Click **Run and Snapshot** to check that your setup script works. Upon success a snapshot of your environment will be created.

Environment Snapshots

[Section titled "Environment Snapshots"](#)

After you click **Run and Snapshot** your environment setup script will be run and you will see the results. Upon success, a snapshot of your environment is taken. This snapshot will be used for future Jules tasks started from this repository. This is especially useful for complex environments with long setup times.

Validation tips

[Section titled "Validation tips"](#)

- You can check installed versions by adding commands like `node -v` to your setup script and clicking **Run to Validate**.
- Always include commands to install packages, run linters, or execute tests.
- Use the validation button to catch errors early.
- Keep your setup lightweight and fast

Running Tasks with Jules

Módulo 1 | Fuente: <https://jules.google/docs/running-tasks/>

Running Tasks with Jules

Once you've logged in and connected GitHub, you're ready to start coding with Jules. This guide walks through the key steps of running a task — from selecting a repo to writing your prompt and setting up notifications.

Choose a repo and branch

[Section titled "Choose a repo and branch"](#)

Jules needs a repo and branch to work on. After logging in:

1. Open the **repo selector** dropdown.
2. Select the repository you'd like Jules to work on.
3. Choose the branch you want to base your changes on.

Jules remembers your last-used repo, so you'll always see the last used repo in the repo selector.

Write a clear prompt

[Section titled "Write a clear prompt"](#)

Jules works best when your prompt is specific and scoped. Use plain language — no need for perfect grammar or code.

Attach visual context

[Section titled "Attach visual context"](#)

You can upload images when initiating a task in Jules. Visuals like UI mockups, front-end glitches, screenshots, diagrams, or inspiration help clarify context when used with your natural language prompt. How to use it:

1. Drag-and-drop or browse to upload one or more images at task creation.
2. Total upload size must stay under 5MB.
3. Supported formats: PNG and JPEG.
4. Images are only attachable during initial task setup; not yet available in follow-up prompts.

Jules ingests these visuals to inform its understanding and output. They won't be embedded in code or committed to your repository. If you want visual assets included in code, upload them to your repo separately.

✔ Good prompts

- Add a loading spinner while `fetchUserProfile` runs
- Fix the 500 error while submitting the feedback form
- Document the `useCache` hook with JSDoc

✘ Avoid

- Fix everything
- Optimize code
- Make this better

If Jules needs more clarity, it will ask for feedback before writing code.

Watching Jules work

[Section titled "Watching Jules work"](#)

Once the plan is approved, Jules will start coding.

You will see:

- An **activity feed** as each step completes
- Inline explanations of each change
- A **mini diff** preview for each file

Use the **diff editor** for a full view across all files.

Final summary & branch creation

[Section titled "Final summary & branch creation"](#)

When Jules finishes a task, it provides a summary of everything it accomplished.

-  **Files changed**
-  **Total runtime**
-  **Lines of code added/changed/removed**
-  **Option to create branch and commit message**

You can click **Create branch** to push the changes. Note that:

- You are the branch owner
- Jules appears as the commit author
- You can open a PR from this branch in GitHub

Giving feedback mid-task

[Section titled "Giving feedback mid-task"](#)

You can send feedback to Jules while it's working:

- Type directly into the chat box
- Ask Jules to change its approach, revise code, or clarify logic
- Jules will respond and, if needed, replan or revise the task

You can intervene at any time, you're in control.

Pausing Jules

[Section titled "Pausing Jules"](#)

You can pause Jules at any time by clicking the **"pause"**.

When Jules is paused it won't do any work, and will wait for your next set of instructions. You can prompt it again, unpause it, or delete the task.

Starting tasks from GitHub Issues

[Section titled "Starting tasks from GitHub Issues"](#)

You can start a task from a **GitHub issue** by applying the label "jules" (case insensitive). Make sure that the Jules GitHub app is authorized to access the repo.

Select an issue, then click the gear icon next to "Labels". Then add the label "jules" (case insensitive) to the issue.

Soon, you will see Jules comment automatically on the issue. When Jules is finished with the issue, it will provide a link to the pull request where you can review its work.

Scheduled Tasks

Módulo 1 | Fuente: <https://jules.google/docs/scheduled-tasks/>

Scheduled Tasks

Jules allows you to set up recurring tasks ensuring continuous execution of routine codebase maintenance, monitoring, or updates without the need for manual re-prompting.

Creating a Scheduled Task

[Section titled "Creating a Scheduled Task"](#)

You can convert any standard task into a recurring one directly from the task input field.

1. Navigate to the main **Task Input** field.
2. Click the **Planning** dropdown located at the bottom right of the input area.
3. Select **Scheduled Task** from the menu.
4. Configure the **Frequency** and **Cadence** (e.g., Daily, Weekly) using the settings that appear at the top of the input field.
5. Write your task prompt and click **Submit**.

The task is now active and will execute automatically at the specified times.

Managing Scheduled Tasks

[Section titled "Managing Scheduled Tasks"](#)

Once a task is created, you can view and manage it within your dashboard.

To find your tasks, look for the **Scheduled** tab located immediately **under the text field** in the dashboard. From this view, you can:

- **Monitor Status:** See when the task last ran and when it is scheduled to run next.
- **Delete:** Remove the scheduled task permanently.

Editing an existing scheduled task is not currently supported. You will need to delete the task and create a new one.

Common Use Cases

[Section titled "Common Use Cases"](#)

Scheduled tasks are best used for maintenance jobs that require consistency but minimal human oversight. We've added three templated scheduled tasks that our friends at [Stitch](#) (Google's AI Design tool) uses. They include:

Use Case	Description
Performance	A performance-obsessed agent who makes the codebase faster, one optimization at a time.
Design	UX-focused agent who adds small touches of delight and accessibility to the user interface.
Security	A security-focused agent who protects the codebase from vulnerabilities and security risks.

Suggested Tasks

Módulo 1 | Fuente: <https://jules.google/docs/suggested-tasks/>

Suggested Tasks

Jules can autonomously scan your codebase to identify areas for improvement and propose implementation plans. These are presented as **Suggested Tasks**.

Currently, this experimental feature focuses on identifying and resolving **#TODO** comments left in your code, with more capabilities coming soon.

Enabling Suggested Tasks

[Section titled "Enabling Suggested Tasks"](#)

To receive suggestions, you must explicitly enable "Proactivity" for each repository you want Jules to monitor.

1. Navigate to the **Codebases** field in the left panel.
2. Select the specific repository (codebase) you wish to activate.
3. With the codebase selected, open the **Proactivity** tab in the left panel.
4. Toggle **Proactivity** to **On**.

Once enabled, Jules will immediately begin scanning the codebase. After a few minutes, if a resolvable issue is found, a suggestion will appear for your review.

Note: At launch, you are only able to enable proactivity on up to 5 repositories.

Reviewing and Running Suggestions

[Section titled "Reviewing and Running Suggestions"](#)

When Jules identifies a potential task (such as a simple **#TODO**), it will surface a suggested fix. You will also receive an email periodically when new tasks have been found.

1. Review the suggestion card that appears in your dashboard.
2. Click on the suggestion to see the context and the rationale.
3. If you agree with the plan, click **Start** to have Jules start working on the task.

Feedback and Dismissal

[Section titled "Feedback and Dismissal"](#)

You remain in control of your backlog. If a suggestion is not relevant or not a priority, you can dismiss a suggestion. Jules uses this feedback to better understand your preferences and improve future suggestions.

Configuration & Limits

[Section titled "Configuration & Limits"](#)

- **Scan Frequency:** Jules scans your repositories periodically to look for new suggestions.
- **Repository Limit:** You can currently enable Suggested Tasks on up to **five repositories**.
- **Current Scope:** The feature currently looks for **#TODO comments** that describe resolvable tasks.

Reviewing plans & giving feedback

Módulo 1 | Fuente: <https://jules.google/docs/review-plan/>

Reviewing plans & giving feedback

Once you start a task in Jules, it generates a **plan** before writing any code. This lets you know the direction Jules will take while it works on the task. From here, you can also iterate with Jules on the plan before any code is written.

Reviewing the plan

[Section titled "Reviewing the plan"](#)

After setup (cloning your repo, initializing the VM, and installing dependencies), Jules will present its plan.

You'll see:

- A natural language description of what Jules intends to direction
 - Step-by-step breakdowns
 - Any assumptions or setup steps
1. Open each step by expanding them to read through each one.
 2. Give feedback using the chat input if you need.
 3. Click **approve plan** when you're ready for Jules to begin executing and writing code.

Note: If you navigate away, Jules will eventually auto-approve the plan, which is set on a timer. So there is no need to babysit.

Giving feedback

[Section titled "Giving feedback"](#)

At any point, you can use the chat box to talk to Jules.

- Ask it to revise a step
- Point out something it missed
- Clarify your original request
- Answer its questions, or provide additional information Jules may need

Jules will respond and update its plan if needed.

Reviewing code changes

Módulo 1 | Fuente: <https://jules.google/docs/code/>

Reviewing code changes

Once you approve a plan, Jules begins working inside a virtual machine. As it completes steps, you'll see an activity log with Jules' thoughts and code updates. This page walks you through what to look for and how to respond.

Activity feed

[Section titled "Activity feed"](#)

As Jules works, you'll see a real-time **activity feed** that logs:

- Each step it completes
- Descriptions of what it did
- Any outputs or errors encountered
- Requests for additional information or feedback

This feed gives you insight into Jules' decision-making and progress.

Code diffs

[Section titled "Code diffs"](#)

When Jules changes code, you'll see a **mini diff** directly in the feed. For a more complete view:

- On the right pane, you can view a full screen expanded **diff editor** to see all changes across files
- The **diff editor** only shows files where it modified or added code.
- Expand the diff editor to full screen, or drag the left sidebar of the diff editor to slide it to your preferred width

You can download and copy code that Jules has written from the download and copy icons located in the top right of the diff editor panel. When you copy code, only the updated code will save to your clip board (not the full diff).

This is your central hub for understanding the scope of the changes Jules made to your repo.

Interactive feedback

[Section titled "Interactive feedback"](#)

You can interact with Jules in real-time through the chat box:

- Ask it to revise logic or naming
- Request additional tests or cleanup
- Give corrections like "return an empty string instead of None"

Task summary

[Section titled "Task summary"](#)

When the task completes, Jules provides a final summary which includes:

-  **Files changed**
-  **Total runtime**
-  **Lines of code added/changed**
-  **Branch name and commit message**

Pushing to GitHub

[Section titled "Pushing to GitHub"](#)

Click **Publish branch** or **Publish PR** to push Jules' changes to GitHub:

- Jules will appear as a commit author on the branch, if you manually create a PR from the branch, you will be the PR author
- When publishing a PR, Jules will appear as the creator of the PR

Once a branch is published, you can continue editing the branch, review it as a GitHub PR, or delete it.

Once a PR is created, you can send it for review, or merge the PR into your codebase.

Managing tasks and repos

Módulo 1 | Fuente: <https://jules.google/docs/tasks-repos/>

Managing tasks and repos

To start a new task:

- 1. Click the Jules icon to return to the home screen and a blank prompt input
- 2. Use the + button in the top navigation on the task view
- 3. From within a specific repo view, click **new task**

You can launch multiple tasks at a time to run them simultaneously. If you are viewing an existing task, or repo page when you kick off a new task, that task will load and begin running in the background. You will have to navigate to the task to check for updates, approve the plan, etc.

Each task runs in its own virtual machine and maintains its own logs, environment setup, and code changes.

Pausing or deleting tasks

[Section titled "Pausing or deleting tasks"](#)

Sometimes you need to stop Jules mid-task or clean up old runs.

- To **pause** a task: click the **pause**.
- To **delete** a task: hover over the task in the repo view and click the **trash icon**.

Paused tasks can be resumed later; deleted tasks are removed permanently.

Managing repositories

[Section titled "Managing repositories"](#)

Jules can only access repositories you explicitly allow through GitHub. In the future, Jules will work with more version control systems.

Selecting a repo

[Section titled "Selecting a repo"](#)

If you've already granted Jules access to all your repos:

- Use the dropdown when starting a task to choose any authorized repo
- Enroll new from the left sidebar
- Search for your repo using the **repo selector**

Granting access to more repos

[Section titled "Granting access to more repos"](#)

To give Jules access to more of your repositories:

1. Go to github.com
2. Click your profile photo —> **settings**
3. In the left sidebar, click **applications**
4. Find **Google Labs Jules** and click **configure**
5. Under **repository access**, select additional repos
6. Click **save**

You can also authorize new repositories via the **repo selector** in Jules. Open the repo selector and scroll to the bottom to click on **+Add repository**. This will take you to GitHub, where you can select additional repositories to grant access for Jules.

Repo view

Módulo 1 | Fuente: <https://jules.google/docs/repo/>

Repo view

The repo view gives you a focused workspace for interacting with a specific repository. From here, you can view task history, manage running tasks, and configure your environment.

Accessing the repo view

[Section titled "Accessing the repo view"](#)

There are two ways to open a repo view:

1. Click on any repository name from the sidebar task list
2. Select a repo from the task picker, then click into it after starting a task

This opens a view scoped to that repository, including task history and available actions.

Viewing tasks by repo

[Section titled "Viewing tasks by repo"](#)

Inside the repo view, you'll see a list of all tasks associated with that repo:

- **Running** tasks are listed at the top
- **Completed** tasks include full logs and diffs
- **Failed** or **waiting** tasks are clearly labeled

You can click on any task to reopen it, review the plan, or continue feedback.

Starting a new task

[Section titled "Starting a new task"](#)

From the repo view, you can start a new task scoped to that repository:

- Click **new tasks** in the top right corner
- This will preselect the repo and branch
- Enter your prompt and optimal setup script as usual

Errors and failures

Módulo 1 | Fuente: <https://jules.google/docs/errors/>

Errors and failures

Even though Jules automates many parts of the development process, things can go wrong such as environment misconfigurations to task failures. This page covers how Jules reports errors, what it does automatically, and how you can debug issues.

How errors are reported

[Section titled "How errors are reported"](#)

Jules surfaces errors in two key places:

- The **activity feed**, where the step failed is logged
- A **notification badge** (red dot) in the UI

These errors can show up whether the task has failed completely or simply requires your intervention.

Automatic retry behavior

[Section titled "Automatic retry behavior"](#)

Jules will attempt to retry failed steps automatically when possible. For example:

- Network hiccups
- Transient install errors
- Slow dependency resolutions

After multiple retries, if the problem persists, the task will be marked as **failed**.

Common causes of failure

[Section titled "Common causes of failure"](#)

The most frequent issues are:

- Incomplete or missing **environment setup scripts**
- Prompts that are too vague or overly broad
- Repos with unusual or nonstandard build systems
- Long-running processes (like `npm run dev`) included in setup

Debugging environment setup

[Section titled "Debugging environment setup"](#)

You can retry a task by:

- Clicking **rerun** from the task summary view
- Modifying your setup script or prompt before restarting the task

Make sure to address any specific feedback from the failure logs.

Limits and Plans

Módulo 1 | Fuente: <https://jules.google/docs/usage-limits/>

Limits and Plans

Jules Plans

[Section titled "Jules Plans"](#)

Jules is available with different plans designed to meet your specific needs, from individual exploration to intensive, professional development. For our initial launch, paid plans are accessed through a Google One subscription.

Note on Plan Availability

Paid Jules plans are accessed via a Google AI Plans subscription, which is currently available only for individual Google Accounts (ending in @gmail.com).

We are actively working on providing upgrade paths for other user types. In the meantime, If you are a business power user and need more access to Jules, please fill out [this interest form](#) and we will get back to you.

Plan Comparison

[Section titled "Plan Comparison"](#)

The table below outlines the key differences between the available plans. All plans have access to the same powerful Gemini models, with different number of task limits. These offerings will evolve, with more advanced models and features planned in the future.

Plan	Jules	Jules in Pro	Jules in Ultra
Best For	Evaluating Jules on real work	Daily coding with higher intensity	Power users & agent-heavy workflows
Daily Tasks (rolling 24 hours)	15	100	300
Concurrent Tasks	3	15	60
Model Access	Gemini 2.5 Pro	Higher access to the latest model (starting with Gemini 3 Pro)	Priority access to the latest model (starting with Gemini 3 Pro)

How to Upgrade

[Section titled "How to Upgrade"](#)

You can access the powerful features of our paid plans by subscribing to a corresponding Google AI plan.

Jules in Pro

[Section titled "Jules in Pro"](#)

Jules in Pro is included as a benefit of the [Google AI Pro plan](#). Upgrading gives you significantly higher task limits, perfect for professional development workflows.

Jules in Ultra

[Section titled "Jules in Ultra"](#)

Jules in Ultra is included as a benefit of the [Google AI Ultra plan](#). This plan provides our highest limits for the most demanding, large-scale projects and workflows.

Frequently Asked Questions

[Section titled "Frequently Asked Questions"](#)

Why are paid plans only available for @gmail.com accounts right now?

For our initial launch, we are launching in the Google AI Plans, which currently serves individual Google Accounts. We are working to quickly support other upgrade paths for users who have enterprise or Workspace accounts.

Will the features and limits in these plans change over time?

We may adjust limits and features as we learn how people are using the product. Our goal is to continue adding value, especially to the Pro and Ultra tiers, with differentiators like access to more advanced models, queue prioritization, more planned for the future.

If my Google One plan is shared with my family, are my Jules task limits shared?

For this launch, task limits are not shared or pooled. Each member of a shared family plan is treated as an individual user with their own set of task limits.

What happens if I use all my tasks for the day?

Once you reach your daily task limit (rolling 24 hour window), you will not be able to trigger new tasks until the limit resets. When you are at your limit, you may see calls to action within the product prompting you to upgrade to a higher plan.

I have a Google AI plan, but I can't access Jules. Why?

Jules has a stricter age requirement than some Google One plans. To use Jules, you must be 18 years of age or older. Therefore, some members of a Google One plan who are under 18 may not have access to Jules benefits, even if the plan itself is active

Can I still access my tasks when I hit a limit?

When you hit a limit:

- The **new task** button will be disabled
- A tooltip or error message will explain the reason
- You can continue reviewing or managing existing tasks
- Task history and feedback are unaffected

Feedback and support

Módulo 1 | Fuente: <https://jules.google/docs/feedback/>

Feedback and support

Jules is in active development, and your feedback directly shapes what we build next. We want to hear from you whether you've found a bug, want to suggest an improvement, or have an idea for a feature.

Giving feedback in the UI

[Section titled "Giving feedback in the UI"](#)

You can send feedback directly from within Jules:

1. Click the **feedback** button in top the navigation bar
2. Select 👍 or 👎
3. Fill out the short form and submit

Your feedback goes directly to the Google Labs team.

What happens after you submit feedback

[Section titled "What happens after you submit feedback"](#)

Your feedback is reviewed by the Jules product and engineering team regularly. We use it to:

- Prioritize new features and fixes
- Understand friction points in your workflow
- Identify common patterns and corner cases

We won't be able to reply to every submission, but we do read all of them.

Feature requests

[Section titled "Feature requests"](#)

Got something you'd love Jules to support? Tell us:

- What you were trying to do
- What got in your way
- How you imagine it could work

Clear, real-world examples are especially helpful.

Thank you for helping us make Jules better.

FAQ

Módulo 1 | Fuente: <https://jules.google/docs/faq/>

FAQ

What is Jules?

[Section titled "What is Jules?"](#)

Jules is a software coding agent that helps you fix bugs, add documentation, update your app, and implement new features. It integrates with GitHub and works autonomously — meaning you can submit a task, go do something else, and return when it's done. Jules is currently in Public Beta.

Is Jules available without cost?

[Section titled "Is Jules available without cost?"](#)

Jules has multiple plans to fit your development needs, including a plan available to all users without cost. See the [Limits and Plans](#) section for more details.

How does Jules work under the hood?

[Section titled "How does Jules work under the hood?"](#)

Each task runs in a fresh virtual machine where Jules clones your repo, installs dependencies, and makes changes based on your prompt. You can provide setup scripts to ensure your project builds and tests correctly.

How does Jules run code, and what should I know about security?

[Section titled "How does Jules run code, and what should I know about security?"](#)

When you run code in Jules, it's executed in a secure, cloud-based virtual machine (VM) with internet access. While this gives you powerful tools to test, build, and debug in context, it's important to treat the environment with the same security precautions you would for any public or shared compute surface. If you're not sure whether something is safe to run, we recommend reviewing it carefully (including non-code components). Jules is a large language model based system which operates on both the code and non-code files in a repository.

You are responsible for the code you run. That means:

- **Don't commit secrets** (like API keys, tokens, or credentials) to your repo, especially if you're pulling code into Jules from a Git provider.
- **Avoid known security vulnerabilities** in your dependencies or scripts. Consider following [GitHub's Quickstart](#) for securing your repository if you're using GitHub.
- **Be cautious with third-party packages or shell commands** that could compromise your system, your data, or others.

Can Jules run long-lived commands like `npm run dev`?

[Section titled "Can Jules run long-lived commands like npm run dev?"](#)

No. Long-running processes like dev servers or watch scripts aren't currently supported in setup scripts. Use discrete install/test commands instead.

What languages does Jules support?

[Section titled "What languages does Jules support?"](#)

Jules is language agnostic but works best with projects that use:

- JavaScript/TypeScript
- Python
- Go
- Java
- Rust

Support depends on what's installed on the VM and the clarity of your environment setup script.

Can I leave Jules while its working?

[Section titled "Can I leave Jules while its working?"](#)

Yes! Jules is designed to be autonomous. You can leave the app after submitting a task. Make sure to [enable notifications](#) so you'll be alerted when a plan is ready or a task completes.

How do I provide feedback or report a bug?

[Section titled "How do I provide feedback or report a bug?"](#)

Click the [feedback](#) button in the bottom left of the Jules UI. No account or tracking system required — feedback goes straight to the team.

What happens if a task fails?

[Section titled "What happens if a task fails?"](#)

Jules will retry automatically. If it continues to fail, it will mark the task as failed and notify you. Common causes include broken setup scripts or vague prompts.

You can revise and rerun the task once you've addressed the issue.

How many tasks can I run?

[Section titled "How many tasks can I run?"](#)

Limits are listed [here](#).

Can I change which repos Jules has access to?

[Section titled "Can I change which repos Jules has access to?"](#)

Yes. Go to your GitHub settings:

1. Click your profile —> **settings**
2. Select **applications** in the sidebar
3. Find **Google Labs Jules** and click **configure**
4. Adjust repository access

Then refresh Jules and your new repos will appear. [Learn more about managing tasks and repositories.](#)

Does Jules train on private repos?

[Section titled "Does Jules train on private repos?"](#)

No. Jules does **not** train on private repository content. Privacy is a core principle for Jules, and we do not use your private repositories to train models. [Learn more](#) about how your data is used to improve Jules.

Contact

Módulo 1 | Fuente: <https://jules.google/docs/contact/>

Contact

If you have any questions, feedback, or need assistance, please join our [Discord](#) or follow us on [X](#).

Módulo 2: Referencia de la REST API

Quickstart

Módulo 2 | Fuente: <https://jules.google/docs/api/reference/>

Quickstart

The Jules REST API lets you programmatically access Jules's capabilities to automate and enhance your software development lifecycle. You can use the API to create custom workflows, automate tasks like bug fixing and code reviews, and embed Jules's intelligence directly into the tools you use every day, such as Slack, Linear, and GitHub.

The Jules REST API is in an alpha release, which means it is experimental. Be aware that we may change specifications, API keys, and definitions as we work toward stabilization. In the future, we plan to maintain at least one stable and one experimental version.

Authentication

[Section titled "Authentication"](#)

To get started with the Jules REST API, you'll need an API key.

Generate Your API Key

[Section titled "Generate Your API Key"](#)

In the Jules web app, go to the [Settings](#) page to create a new API key. You can have at most 3 API keys at a time.

Use Your API Key

[Section titled "Use Your API Key"](#)

To authenticate your requests, pass the API key in the `X-Goog-API-Key` header of your API calls.

Keep your API keys secure. Don't share them or embed them in public code. For your protection, any API keys found to be publicly exposed will be [automatically disabled](#) to prevent abuse.

API concepts

[Section titled "API concepts"](#)

The Jules REST API is built around a few core resources. Understanding these will help you use the API effectively.

- **Source** — An input source for the agent (e.g., a GitHub repository). Before using a source using the API, you must first [install the Jules GitHub app](#) through the Jules web app.
- **Session** — A continuous unit of work within a specific context, similar to a chat session. A session is initiated with a prompt and a source.

- **Activity** — A single unit of work within a Session. A Session contains multiple activities from both the user and the agent, such as generating a plan, sending a message, or updating progress.

Your first API call

[Section titled "Your first API call"](#)

We'll walk through creating your first session with the Jules REST API using curl.

1. List your available sources

[Section titled "List your available sources"](#)

First, you need to find the name of the source you want to work with (e.g., your GitHub repo). This command will return a list of all sources you have connected to Jules.

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sources
```

The response will look something like this:

```
{
  "sources": [
    {
      "name": "sources/github/bobalover/boba",
      "id": "github/bobalover/boba",
      "githubRepo": {
        "owner": "bobalover",
        "repo": "boba"
      }
    }
  ],
  "nextPageToken": "github/bobalover/boba-web"
}
```

2. Create a new session

[Section titled "Create a new session"](#)

Now, create a new session. You'll need the source name from the previous step. This request tells Jules to create a boba app in the specified repository.

```
curl 'https://jules.googleapis.com/v1alpha/sessions' \
-X POST \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $JULES_API_KEY" \
-d '{
  "prompt": "Create a boba app!",
  "sourceContext": {
    "source": "sources/github/bobalover/boba",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  },
  "automationMode": "AUTO_CREATE_PR",
  "title": "Boba App"
}'
```

The `automationMode` field is optional. By default, no PR will be automatically created.

The immediate response will look something like this:

```
{
  "name": "sessions/31415926535897932384",
  "id": "31415926535897932384",
  "title": "Boba App",
  "sourceContext": {
    "source": "sources/github/bobalover/boba",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  },
  "prompt": "Create a boba app!"
}
```

You can poll the latest session information using `GetSession` or `ListSessions`. For example, if a PR was automatically created, you can see the PR in the session output:

```
{
  "name": "sessions/31415926535897932384",
  "id": "31415926535897932384",
  "title": "Boba App",
  "sourceContext": {
    "source": "sources/github/bobalover/boba",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  },
  "prompt": "Create a boba app!",
  "outputs": [
    {
      "pullRequest": {
        "url": "https://github.com/bobalover/boba/pull/35",
        "title": "Create a boba app",
        "description": "This change adds the initial implementation of a boba app."
      }
    }
  ]
}
```

By default, sessions created through the API will have their plans automatically approved. If you want to create a session that requires explicit plan approval, set the `requirePlanApproval` field to `true`.

3. List sessions

[Section titled "List sessions"](#)

You can list your sessions as follows:

```
curl 'https://jules.googleapis.com/v1alpha/sessions?pageSize=5' \
-H "x-goog-api-key: $JULES_API_KEY"
```

4. Approve a plan

[Section titled "Approve a plan"](#)

If your session requires explicit plan approval, you can approve the latest plan as follows:

```
curl 'https://jules.googleapis.com/v1alpha/sessions/SESSION_ID:approvePlan' \
-X POST \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $JULES_API_KEY"
```

5. Interact with the agent

[Section titled "Interact with the agent"](#)

To list activities in a session:

```
curl 'https://jules.googleapis.com/v1alpha/sessions/SESSION_ID/activities?pageSize=30' \
-H "x-goog-api-key: $JULES_API_KEY"
```

To send a message to the agent:

```
curl 'https://jules.googleapis.com/v1alpha/sessions/SESSION_ID:sendMessage' \
-X POST \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $JULES_API_KEY" \
-d '{
  "prompt": "Can you make the app corgi themed?"
}'
```

The response will be empty because the agent will send its response in the next activity. To see the agent's response, list the activities again.

Next steps

[Section titled "Next steps"](#)

[API Reference](#) Complete documentation for all Jules REST API endpoints, request/response formats, and data types.

API Reference

Módulo 2 | Fuente: <https://jules.google/docs/api/reference/overview>

API Reference

The Jules REST API allows you to programmatically create and manage coding sessions, monitor progress, and retrieve results. This reference documents all available endpoints, request/response formats, and data types.

Base URL

Section titled "Base URL"

All API requests should be made to:

```
https://jules.googleapis.com/v1alpha
```

Authentication

Section titled "Authentication"

The Jules REST API uses API keys for authentication. Get your API key from jules.google.com/settings.

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sessions
```

[Authentication Guide](#) Detailed setup instructions for API key authentication.

Endpoints

Section titled "Endpoints"

- [Sessions](#) Create and manage coding sessions. Sessions represent a unit of work where Jules executes tasks on your codebase.
- [Activities](#) Monitor session progress through activities. Each activity represents an event like plan generation, messages, or completion.
- [Sources](#) List and retrieve connected repositories. Sources represent GitHub repositories that Jules can work with.
- [Types](#) Reference for all data types used in the API including Session, Activity, Plan, Artifact, and more.

Common Patterns

Section titled "Common Patterns"

Pagination

Section titled "Pagination"

List endpoints support pagination using `pageSize` and `pageToken` parameters:

```
# First page
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sessions?pageSize=10"

# Next page (using token from previous response)
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sessions?pageSize=10&pageToken=NEXT_PAGE_TOKEN"
```

Resource Names

Section titled "Resource Names"

Resources use hierarchical names following Google API conventions:

- Sessions: `sessions/{sessionId}`
- Activities: `sessions/{sessionId}/activities/{activityId}`
- Sources: `sources/{sourceId}`

Error Handling

[Section titled "Error Handling"](#)

The API returns standard HTTP status codes:

Status	Description
200	Success
400	Bad request - invalid parameters
401	Unauthorized - invalid or missing token
403	Forbidden - insufficient permissions
404	Not found - resource doesn't exist
429	Rate limited - too many requests
500	Server error

Error responses include a JSON body with details:

```
{
  "error": {
    "code": 400,
    "message": "Invalid session ID format",
    "status": "INVALID_ARGUMENT"
  }
}
```

Authentication

Módulo 2 | Fuente: <https://jules.google/docs/api/reference/authentication>

Authentication

The Jules REST API uses API keys for authentication. You'll need a valid API key to make API requests.

Getting Your API Key

[Section titled "Getting Your API Key"](#)

1. Go to jules.google.com/settings
2. Find the **API Key** section
3. Click **Generate API Key** (or copy your existing key)
4. Store the key securely — it won't be shown again

Keep your API key secret. Don't commit it to version control or share it publicly.

Using Your API Key

[Section titled "Using Your API Key"](#)

Include the API key in the `x-goog-api-key` header with every request:

```
curl -H "x-goog-api-key: YOUR_API_KEY" \
https://jules.googleapis.com/v1alpha/sessions
```

Environment Variable (Recommended)

[Section titled "Environment Variable \(Recommended\)"](#)

Store your API key in an environment variable:

```
export JULES_API_KEY="your-api-key-here"
```

Then use it in requests:

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sessions
```

Example: Create a Session

[Section titled "Example: Create a Session"](#)

```
curl -X POST \
-H "x-goog-api-key: $JULES_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "prompt": "Add unit tests for the utils module",
  "sourceContext": {
    "source": "sources/github-owner-repo",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  }
}' \
https://jules.googleapis.com/v1alpha/sessions
```

Example: List Sessions

[Section titled "Example: List Sessions"](#)

...

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sessions
```

Troubleshooting

[Section titled "Troubleshooting"](#)

"API key not valid"

[Section titled "'API key not valid'"](#)

- Verify you copied the entire key without extra spaces
- Check that the key hasn't been revoked in [settings](#)
- Generate a new key if needed

"Permission denied"

[Section titled "'Permission denied'"](#)

- Verify your account has access to Jules
- Check that you have access to the requested resources (sessions, sources)

"Quota exceeded"

[Section titled "'Quota exceeded'"](#)

- You may have hit rate limits

Sessions

Módulo 2 | Fuente: <https://jules.google/docs/api/reference/sessions>

Sessions

Sessions are the core resource in the Jules REST API. A session represents a unit of work where Jules executes a coding task on your repository.

Create a Session

Section titled "Create a Session"

POST `/v1alpha/sessions`

Creates a new session to start a coding task.

Request Body

Section titled "Request Body"

`prompt` required string

The task description for Jules to execute.

`title` string

Optional title for the session. If not provided, the system will generate one.

`sourceContext` [SourceContext](#)

The source repository and branch context for this session. Optional for repoless sessions.

`requirePlanApproval` boolean

If true, plans require explicit approval before execution. If not set, plans are auto-approved.

`automationMode` string

Automation mode. Use 'AUTO_CREATE_PR' to automatically create pull requests when code changes are ready.

Example Request

Section titled "Example Request"

```
curl -X POST \
-H "x-goog-api-key: $JULES_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "prompt": "Add comprehensive unit tests for the authentication module",
  "title": "Add auth tests",
  "sourceContext": {
    "source": "sources/github-myorg-myrepo",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  },
  "requirePlanApproval": true
}' \
https://jules.googleapis.com/v1alpha/sessions
```

Response

Section titled "Response"

Returns the created [Session](#) object:

```
{
  "name": "sessions/1234567",
  "id": "abc123",
  "prompt": "Add comprehensive unit tests for the authentication module",
  "title": "Add auth tests",
  "state": "QUEUED",
  "url": "https://jules.google.com/session/abc123",
  "createTime": "2024-01-15T10:30:00Z",
  "updateTime": "2024-01-15T10:30:00Z"
}
```

List Sessions

[Section titled "List Sessions"](#)

GET `/v1alpha/sessions`

Lists all sessions for the authenticated user.

Query Parameters

[Section titled "Query Parameters"](#)

`pageSize` integer query

Number of sessions to return (1-100). Defaults to 30.

`pageToken` string query

Page token from a previous ListSessions response.

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sessions?pageSize=10"
```

Response

[Section titled "Response"](#)

```
{
  "sessions": [
    {
      "name": "sessions/1234567",
      "id": "abc123",
      "title": "Add auth tests",
      "state": "COMPLETED",
      "createTime": "2024-01-15T10:30:00Z",
      "updateTime": "2024-01-15T11:45:00Z"
    }
  ],
  "nextPageToken": "eyJvZmZzZXQiOjEwEwfQ=="
}
```

Get a Session

[Section titled "Get a Session"](#)

GET `/v1alpha/sessions/{sessionId}`

Retrieves a single session by ID.

Path Parameters

[Section titled "Path Parameters"](#)

`name` required string path

The resource name of the session. Format: sessions/{session}

Pattern: `^sessions/[/]+[a-z0-9-]+$`

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
  https://jules.googleapis.com/v1alpha/sessions/1234567
```

Response

[Section titled "Response"](#)

Returns the full [Session](#) object including outputs if the session has completed:

```
{
  "name": "sessions/1234567",
  "id": "abc123",
  "prompt": "Add comprehensive unit tests for the authentication module",
  "title": "Add auth tests",
  "state": "COMPLETED",
  "url": "https://jules.google.com/session/abc123",
  "createTime": "2024-01-15T10:30:00Z",
  "updateTime": "2024-01-15T11:45:00Z",
  "outputs": [
    {
      "pullRequest": {
        "url": "https://github.com/myorg/myrepo/pull/42",
        "title": "Add auth tests",
        "description": "Added unit tests for authentication module"
      }
    }
  ]
}
```

Delete a Session

Section titled "Delete a Session"

DELETE /v1alpha/sessions/{sessionId}

Deletes a session.

Path Parameters

Section titled "Path Parameters"

`name` required string path

The resource name of the session to delete. Format: sessions/{session}

Pattern: `^sessions/[/]+[a-z0-9-]*$`

Example Request

Section titled "Example Request"

```
curl -X DELETE \
-H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sessions/1234567
```

Response

Section titled "Response"

Returns an empty response on success.

Send a Message

Section titled "Send a Message"

POST /v1alpha/sessions/{sessionId}:sendMessage

Sends a message from the user to an active session.

Use this endpoint to provide feedback, answer questions, or give additional instructions to Jules during an active session.

Path Parameters

Section titled "Path Parameters"

`session` required string path

The resource name of the session. Format: sessions/{session}

Pattern: `^sessions/[/]+[a-z0-9-]*$`

Request Body

Section titled "Request Body"

`prompt` required string

The message to send to the session.

Example Request

Section titled "Example Request"

```
curl -X POST \
  -H "x-goog-api-key: $JULES_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Please also add integration tests for the login flow"
  }' \
  https://jules.googleapis.com/v1alpha/sessions/1234567:sendMessage
```

Response

[Section titled "Response"](#)
Returns an empty [SendMessageResponse](#) on success.

Approve a Plan

[Section titled "Approve a Plan"](#)
POST `/v1alpha/sessions/{sessionId}:approvePlan`
Approves a pending plan in a session.

This endpoint is only needed when `requirePlanApproval` was set to `true` when creating the session.

Path Parameters

[Section titled "Path Parameters"](#)
`sessionId` required string path
The resource name of the session. Format: sessions/{session}

Pattern: `^sessions/[/]+[a-z0-9-]*$`

Example Request

[Section titled "Example Request"](#)

```
curl -X POST \
  -H "x-goog-api-key: $JULES_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{}' \
  https://jules.googleapis.com/v1alpha/sessions/1234567:approvePlan
```

Response

[Section titled "Response"](#)
Returns an empty [ApprovePlanResponse](#) on success.

Session States

[Section titled "Session States"](#)
Sessions progress through the following states:

State	Description
QUEUED	Session is waiting to be processed
PLANNING	Jules is analyzing the task and creating a plan
AWAITING_PLAN_APPROVAL	Plan is ready and waiting for user approval
AWAITING_USER_FEEDBACK	Jules needs additional input from the user
IN_PROGRESS	Jules is actively working on the task
PAUSED	Session is paused
COMPLETED	Task completed successfully
FAILED	Task failed to complete

Activities

Módulo 2 | Fuente: <https://jules.google/docs/api/reference/activities>

Activities

Activities represent events that occur during a session. Use the Activities API to monitor progress, retrieve messages, and access artifacts like code changes.

List Activities

Section titled "List Activities"

GET `/v1alpha/sessions/{sessionId}/activities`

Lists all activities for a session.

Path Parameters

Section titled "Path Parameters"

`parent` required string path

The parent session. Format: sessions/{session}

Pattern: `^sessions/[^\s]+$`

Query Parameters

Section titled "Query Parameters"

`pageSize` integer query

Number of activities to return (1-100). Defaults to 50.

`pageToken` string query

Page token from a previous ListActivities response.

Example Requests

Section titled "Example Requests"



```
curl -H "x-goog-api-key: $JULES_API_KEY" \
      "https://jules.googleapis.com/v1alpha/sessions/1234567/activities?pageSize=20"
```



```
# Define the endpoint to prevent overflow
ENDPOINT="https://jules.googleapis.com/v1alpha/sessions/1234567/activities"
TIMESTAMP="2026-01-17T00:03:53.137240Z"

curl -H "x-goog-api-key: $JULES_API_KEY" \
      "$ENDPOINT?createTime=$TIMESTAMP"
```

Response

Section titled "Response"

Get an Activity

Section titled “Get an Activity”

GET /v1alpha/sessions/{sessionId}/activities/{activityId}

Retrieves a single activity by ID.

Path Parameters

Section titled "Path Parameters"

name required string path

The resource name of the activity. Format: sessions/{session}/activities/{activity}

Pattern: `^sessions/[^/]+/activities/[^/]+$`

Example Request

Section titled "Example Request"

Response

Section titled "Response"

Returns the full [Activity](#) object:

```
{
  "name": "sessions/1234567/activities/act2",
  "id": "act2",
  "originator": "agent",
  "description": "Code changes ready",
  "createTime": "2024-01-15T11:00:00Z",
  "artifacts": [
    {
      "changeSet": {
        "source": "sources/github-myorg-myrepo",
        "gitPatch": {
          "baseCommitId": "a1b2c3d4",
          "unidiffPatch": "diff --git a/tests/auth.test.js...",
          "suggestedCommitMessage": "Add unit tests for authentication module"
        }
      }
    }
  ]
}
```

Activity Types

[Section titled "Activity Types"](#)

Activities have different types based on what occurred. Each activity will have exactly one of these event fields populated:

Plan Generated

[Section titled "Plan Generated"](#)

Indicates Jules has created a plan for the task:

```
{
  "planGenerated": {
    "plan": {
      "id": "plan1",
      "steps": [
        { "id": "step1", "index": 0, "title": "Step title", "description": "Details" }
      ],
      "createTime": "2024-01-15T10:31:00Z"
    }
  }
}
```

Plan Approved

[Section titled "Plan Approved"](#)

Indicates a plan was approved (by user or auto-approved):

```
{
  "planApproved": {
    "planId": "plan1"
  }
}
```

User Messaged

[Section titled "User Messaged"](#)

A message from the user:

```
{
  "userMessaged": {
    "userMessage": "Please also add integration tests"
  }
}
```

Agent Messaged

[Section titled "Agent Messaged"](#)

A message from Jules:

```
{
  "agentMessaged": {
    "agentMessage": "I've completed the unit tests. Would you like me to add integration tests as well?"
  }
}
```

Progress Updated

[Section titled "Progress Updated"](#)

A status update during execution:

```
{
  "progressUpdated": {
    "title": "Writing tests",
    "description": "Creating test cases for login functionality"
  }
}
```

Session Completed

[Section titled "Session Completed"](#)

The session finished successfully:

```
{
  "sessionCompleted": {}
}
```

Session Failed

[Section titled "Session Failed"](#)

The session encountered an error:

```
{
  "sessionFailed": {
    "reason": "Unable to install dependencies"
  }
}
```

Artifacts

[Section titled "Artifacts"](#)

Activities may include artifacts—outputs produced during execution:

Code Changes (ChangeSet)

[Section titled "Code Changes \(ChangeSet\)"](#)

```
{
  "artifacts": [
    {
      "changeSet": {
        "source": "sources/github-myorg-myrepo",
        "gitPatch": {
          "baseCommitId": "a1b2c3d4e5f6",
          "unidiffPatch": "diff --git a/src/auth.js b/src/auth.js\n...",
          "suggestedCommitMessage": "Add authentication tests"
        }
      }
    }
  ]
}
```

Bash Output

[Section titled "Bash Output"](#)

```
{
  "artifacts": [
    {
      "bashOutput": {
        "command": "npm test",
        "output": "All tests passed (42 passing)",
        "exitCode": 0
      }
    }
  ]
}
```

Media

[Section titled "Media"](#)

```
{
  "artifacts": [
    {
      "media": {
        "mimeType": "image/png",
        "data": "base64-encoded-data..."
      }
    }
  ]
}
```

Sources

Módulo 2 | Fuente: <https://jules.google/docs/api/reference/sources>

Sources

Sources represent repositories connected to Jules. Currently, Jules supports GitHub repositories. Use the Sources API to list available repositories and get details about specific sources.

Sources are created when you connect a GitHub repository to Jules through the web interface. The API currently only supports reading sources, not creating them.

List Sources

[Section titled "List Sources"](#)

GET `/v1alpha/sources`

Lists all sources (repositories) connected to your account.

Query Parameters

[Section titled "Query Parameters"](#)

`pageSize` integer query

Number of sources to return (1-100). Defaults to 30.

`pageToken` string query

Page token from a previous ListSources response.

`filter` string query

Filter expression based on AIP-160. Example: 'name=sources/source1 OR name=sources/source2'

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sources?pageSize=10"
```

Response

[Section titled "Response"](#)

```
{
  "sources": [
    {
      "name": "sources/github-myorg-myrepo",
      "id": "github-myorg-myrepo",
      "githubRepo": {
        "owner": "myorg",
        "repo": "myrepo",
        "isPrivate": false,
        "defaultBranch": {
          "displayName": "main"
        },
        "branches": [
          { "displayName": "main" },
          { "displayName": "develop" },
          { "displayName": "feature/auth" }
        ]
      }
    },
    {
      "name": "sources/github-myorg-another-repo",
      "id": "github-myorg-another-repo",
      "githubRepo": {
        "owner": "myorg",
        "repo": "another-repo",
        "isPrivate": true,
        "defaultBranch": {
          "displayName": "main"
        },
        "branches": [
          { "displayName": "main" }
        ]
      }
    }
  ],
  "nextPageToken": "eyJvZmZzZXQiOjEwfQ=="
}
```

Filtering

[Section titled "Filtering"](#)

Use the `filter` parameter to retrieve specific sources:

```
# Get a specific source
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sources?filter=name%3Dsources%2Fgithub-myorg-myrepo"

# Get multiple sources
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sources?filter=name%3Dsources%2Fsource1%20R%20name%3Dsources%2Fsource2"
```

Get a Source

[Section titled "Get a Source"](#)

GET `/v1alpha/sources/{sourceId}`

Retrieves a single source by ID.

Path Parameters

[Section titled "Path Parameters"](#)

`name` required string path

The resource name of the source. Format: sources/{source}

Pattern: `^sources/.*$`

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
  https://jules.googleapis.com/v1alpha/sources/github-myorg-myrepo
```

Response

[Section titled "Response"](#)

Returns the full [Source](#) object:

```
{
  "name": "sources/github-myorg-myrepo",
  "id": "github-myorg-myrepo",
  "githubRepo": {
    "owner": "myorg",
    "repo": "myrepo",
    "isPrivate": false,
    "defaultBranch": {
      "displayName": "main"
    },
    "branches": [
      { "displayName": "main" },
      { "displayName": "develop" },
      { "displayName": "feature/auth" },
      { "displayName": "feature/tests" }
    ]
  }
}
```

Using Sources with Sessions

[Section titled "Using Sources with Sessions"](#)

When creating a session, reference a source using its resource name in the `sourceContext`:

```
curl -X POST \
-H "x-goog-api-key: $JULES_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "prompt": "Add unit tests for the auth module",
  "sourceContext": {
    "source": "sources/github-myorg-myrepo",
    "githubRepoContext": {
      "startingBranch": "develop"
    }
  }
}' \
https://jules.googleapis.com/v1alpha/sessions
```

Use the List Sources endpoint to discover available source names, then use the Get Source endpoint to see available branches before creating a session.

Types Reference

Módulo 2 | Fuente: <https://jules.google/docs/api/reference/types>

Types Reference

This page documents all data types used in the Jules REST API.

Core Resources

[Section titled "Core Resources"](#)

Session

[Section titled "Session"](#)

A session represents a unit of work where Jules executes a coding task.

Session

A session is a contiguous amount of work within the same context.

`name` string

Output only. The full resource name (e.g., 'sessions/{session}').

`id` string

Output only. The session ID.

`prompt` required string

The task description for Jules to execute.

`title` string

Optional title. If not provided, the system generates one.

`state` [SessionState](#)

Output only. Current state of the session.

`url` string

Output only. URL to view the session in the Jules web app.

`sourceContext` required [SourceContext](#)

The source repository and branch context.

`requirePlanApproval` boolean

Input only. If true, plans require explicit approval.

`automationMode` [AutomationMode](#)

Input only. Automation mode for the session.

`outputs` [SessionOutput](#) []

Output only. Results of the session (e.g., pull requests).

`createTime` string (google-datetime)

Output only. When the session was created.

`updateTime` string (google-datetime)

Output only. When the session was last updated.

SessionState

[Section titled "SessionState"](#)

Enum representing the current state of a session:

Value	Description
STATE_UNSPECIFIED	State is unspecified
QUEUED	Session is waiting to be processed
PLANNING	Jules is creating a plan
AWAITING_PLAN_APPROVAL	Plan is ready for user approval
AWAITING_USER_FEEDBACK	Jules needs user input
IN_PROGRESS	Jules is actively working
PAUSED	Session is paused
FAILED	Session failed
COMPLETED	Session completed successfully

AutomationMode

[Section titled "AutomationMode"](#)

Enum for session automation settings:

Value	Description
AUTOMATION_MODE_UNSPECIFIED	No automation (default)
AUTO_CREATE_PR	Automatically create a pull request when code changes are ready

Activity

[Section titled "Activity"](#)

An activity represents a single event within a session.

Activity

An activity is a single unit of work within a session.

`name` string

The full resource name (e.g., 'sessions/{session}/activities/{activity}').

`id` string

Output only. The activity ID.

`originator` string

The entity that created this activity ('user', 'agent', or 'system').

`description` string

Output only. A description of this activity.

`createTime` string (google-datetime)

Output only. When the activity was created.

`artifacts` [Artifact](#) []

Output only. Artifacts produced by this activity.

`planGenerated` [PlanGenerated](#)

A plan was generated.

`planApproved` [PlanApproved](#)

A plan was approved.

`userMessaged` [UserMessaged](#)

The user posted a message.

`agentMessaged` [AgentMessaged](#)

Jules posted a message.

`progressUpdated` [ProgressUpdated](#)

A progress update occurred.

`sessionCompleted` [SessionCompleted](#)

The session completed.

`sessionFailed` [SessionFailed](#)

The session failed.

Source

[Section titled "Source"](#)

A source represents a connected repository.

Source

An input source of data for a session.

`name` string

The full resource name (e.g., 'sources/{source}').

`id` string

Output only. The source ID.

`githubRepo` [GitHubRepo](#)

GitHub repository details.

Plans

[Section titled "Plans"](#)

Plan

[Section titled "Plan"](#)

Plan

A sequence of steps that Jules will take to complete the task.

`id` string

Output only. Unique ID for this plan within a session.

`steps` [PlanStep](#) []

Output only. The steps in the plan.

`createTime` string (google-datetime)

Output only. When the plan was created.

PlanStep

[Section titled "PlanStep"](#)

PlanStep

A single step in a plan.

`id` string

Output only. Unique ID for this step within a plan.

`index` integer (int32)

Output only. 0-based index in the plan.

`title` string

Output only. The title of the step.

`description` string

Output only. Detailed description of the step.

Artifacts

[Section titled "Artifacts"](#)

Artifact

[Section titled "Artifact"](#)

Artifact

A single unit of data produced by an activity.

`changeSet` [ChangeSet](#)

Code changes produced.

`bashOutput` [BashOutput](#)

Command output produced.

`media` [Media](#)

Media file produced (e.g., image, video).

ChangeSet

[Section titled "ChangeSet"](#)

ChangeSet

A set of changes to be applied to a source.

`source` string

The source this change set applies to. Format: sources/{source}

`gitPatch` [GitPatch](#)

The patch in Git format.

GitPatch

[Section titled "GitPatch"](#)

GitPatch

A patch in Git format.

`baseCommitId` string

The commit ID the patch should be applied to.

`unidiffPatch` string

The patch in unified diff format.

`suggestedCommitMessage` string

A suggested commit message for the patch.

BashOutput

[Section titled "BashOutput"](#)

BashOutput

Output from a bash command.

`command` string

The bash command that was executed.

`output` string

Combined stdout and stderr output.

`exitCode` integer (int32)

The exit code of the command.

Media

[Section titled "Media"](#)

Media

A media file output.

`mimeType` string

The MIME type of the media (e.g., 'image/png').

`data` string (byte)
Base64-encoded media data.

GitHub Types

[Section titled "GitHub Types"](#)

GitHubRepo

[Section titled "GitHubRepo"](#)

GitHubRepo

A GitHub repository.

`owner` string
The repository owner (user or organization).

`repo` string
The repository name.

`isPrivate` boolean
Whether the repository is private.

`defaultBranch` [GitHubBranch](#)
The default branch.

`branches` [GitHubBranch](#) []
List of active branches.

GitHubBranch

[Section titled "GitHubBranch"](#)

GitHubBranch

A GitHub branch.

`displayName` string
The branch name.

GitHubRepoContext

[Section titled "GitHubRepoContext"](#)

GitHubRepoContext

Context for using a GitHub repo in a session.

`startingBranch` required string
The branch to start the session from.

Context Types

[Section titled "Context Types"](#)

SourceContext

[Section titled "SourceContext"](#)

SourceContext

Context for how to use a source in a session.

`source` required string
The source resource name. Format: sources/{source}

`githubRepoContext` [GitHubRepoContext](#)
Context for GitHub repositories.

Output Types

[Section titled "Output Types"](#)

SessionOutput

[Section titled "SessionOutput"](#)

SessionOutput

An output of a session.

`pullRequest` [PullRequest](#)

A pull request created by the session.

PullRequest

[Section titled "PullRequest"](#)

PullRequest

A pull request.

`url` string

The URL of the pull request.

`title` string

The title of the pull request.

`description` string

The description of the pull request.

Activity Event Types

[Section titled "Activity Event Types"](#)

PlanGenerated

[Section titled "PlanGenerated"](#)

PlanGenerated

A plan was generated.

`plan` [Plan](#)

The generated plan.

PlanApproved

[Section titled "PlanApproved"](#)

PlanApproved

A plan was approved.

`planId` string

The ID of the approved plan.

UserMessaged

[Section titled "UserMessaged"](#)

UserMessaged

The user posted a message.

`userMessage` string

The message content.

AgentMessaged

[Section titled "AgentMessaged"](#)

AgentMessaged

Jules posted a message.

`agentMessage` string

The message content.

ProgressUpdated

[Section titled "ProgressUpdated"](#)

ProgressUpdated

A progress update occurred.

`title` string

The title of the update.

`description` string

Details about the progress.

SessionCompleted

[Section titled "SessionCompleted"](#)

SessionCompleted

The session completed successfully.

No additional properties.

SessionFailed

[Section titled "SessionFailed"](#)

SessionFailed

The session failed.

`reason` string

The reason for the failure.

Request/Response Types

[Section titled "Request/Response Types"](#)

SendMessageRequest

[Section titled "SendMessageRequest"](#)

SendMessageRequest

Request to send a message to a session.

`prompt` required string

The message to send.

SendMessageResponse

[Section titled "SendMessageResponse"](#)

SendMessageResponse

Response from sending a message.

Empty response on success.

ApprovePlanRequest

[Section titled "ApprovePlanRequest"](#)

ApprovePlanRequest

Request to approve a plan.

Empty request body.

ApprovePlanResponse

[Section titled "ApprovePlanResponse"](#)

ApprovePlanResponse

Response from approving a plan.

Empty response on success.

ListSessionsResponse

[Section titled "ListSessionsResponse"](#)

ListSessionsResponse

Response from listing sessions.

`sessions` [Session](#) []

The list of sessions.

`nextPageToken` string

Token for the next page of results.

ListActivitiesResponse

[Section titled "ListActivitiesResponse"](#)

ListActivitiesResponse

Response from listing activities.

`activities` [Activity](#) []

The list of activities.

`nextPageToken` string

Token for the next page of results.

ListSourcesResponse

[Section titled "ListSourcesResponse"](#)

ListSourcesResponse

Response from listing sources.

`sources` [Source](#) []

The list of sources.

`nextPageToken` string

Token for the next page of results.

Módulo 3: Herramientas CLI de Jules (Jules Tools)

Jules Tools Reference

Módulo 3 | Fuente: <https://jules.google/docs/cli/reference>

Jules Tools Reference

Jules Tools is a lightweight command-line interface (CLI) for interacting with Jules, Google's autonomous AI coding agent. It allows you to manage coding sessions, inspect progress, and integrate Jules into your existing development workflows and scripts directly from your terminal.

Think of Jules Tools as both a command surface and a dashboard for your coding agent, designed to keep you in your flow without needing to switch to a web browser.

Installation

Section titled "Installation"

To get started, install the tool globally using npm or pnpm.

```
npm install -g @google/jules
```

Once installed, the jules command will be available in your terminal.

Authentication

Section titled "Authentication"

Before you can use the tool, you must authenticate with your Google account.

Login

Section titled "Login"

```
jules login
```

This command will open a browser window to guide you through the Google authentication process.

Logout

Section titled "Logout"

To log out from your account:

```
jules logout
```

Usage

Section titled "Usage"

The CLI is built around commands and subcommands. You can get help for any command by using the -h or —help flag.

```
# Get general help
jules help

# Get help for a specific command (e.g., remote)
jules remote --help
```

Global Flags

Section titled "Global Flags"

- h , --help : Displays help information for jules or a specific command.
- theme <string> : Sets the theme for the terminal user interface (TUI). Options are dark (default) or light .

Example: jules --theme light

Available Commands

Section titled "Available Commands"

version

Shows the currently installed version of the Jules Tools CLI.

```
jules version
```

remote

The `remote` command is the primary way to interact with Jules sessions running in the cloud. It has several subcommands.

`remote list` Lists your connected repositories or active sessions.

- `--repo` : Flag to list all repositories connected to Jules.
- `--session` : Flag to list all your remote sessions.

Examples:

```
# List all connected repositories
jules remote list --repo

# List all active and past sessions
jules remote list --session
```

remote new

Creates a new remote session to delegate a task to Jules.

Jules can automatically infer the repository from your current working directory, so you can often omit the `--repo` flag.

- `--repo <repo_name>` : Specifies the repository for the session (e.g., `torvalds/linux` or `.` for the current directory's repo).
- `--session "<prompt>"` : A string describing the task for Jules to perform.
- `--parallel <number>` : Starts multiple parallel sessions to work on the same task.

Example:

```
# Start a new session to write unit tests in the 'torvalds/linux' repo
jules remote new --repo torvalds/linux --session "write unit tests"
```

remote pull

Pulls the results (e.g., code changes) from a completed session.

- `--session <session_id>` : The ID of the session you want to pull.

Example:

```
# Pull the results for session ID 123456
jules remote pull --session 123456
```

completion

Generates an autocompletion script for your shell (e.g., `bash`, `zsh`) to enable tab completion for jules commands.

```
# Generate completion script for bash
jules completion bash
```

Interactive Dashboard (TUI)

[Section titled "Interactive Dashboard \(TUI\)"](#)

For a more interactive, visual experience, you can launch the Terminal User Interface (TUI) by running the `jules` command without any arguments.

```
jules
```

The TUI provides a dashboard view of your sessions, a side-by-side diff viewer for reviewing changes, and guided flows for creating new ones, similar to the web UI.

Practical Examples & Scripting

Módulo 3 | Fuente: <https://jules.google/docs/cli/examples>

Practical Examples & Scripting

Jules Tools is designed to be scriptable and can be composed with other command-line tools.

Below are some examples of Jules Tools in action:

1. Create sessions from a TODO.md file:

Assign each line item from a local TODO.md file as a new session in the current repository.

```
cat TODO.md | while IFS= read -r line; do
  jules remote new --repo . --session "$line"
done
```

2. Create a session from a GitHub Issue:

Pipe the title of the first GitHub issue assigned to you directly into a new Jules session. (Requires the gh and jq CLIs).

```
gh issue list --assignee @me --limit 1 --json title \
| jq -r '[0].title' \
| jules remote new --repo .
```

3. Use Gemini to analyze and assign the hardest issue to Jules: Use the Gemini CLI to analyze your assigned GitHub issues, identify the most tedious one, and pipe its title to Jules.

```
gemini -p "find the most tedious issue, print it verbatim\n$(gh issue list --assignee @me)" \
| jules remote new --repo .
```

Módulo 4: Guías e Integraciones

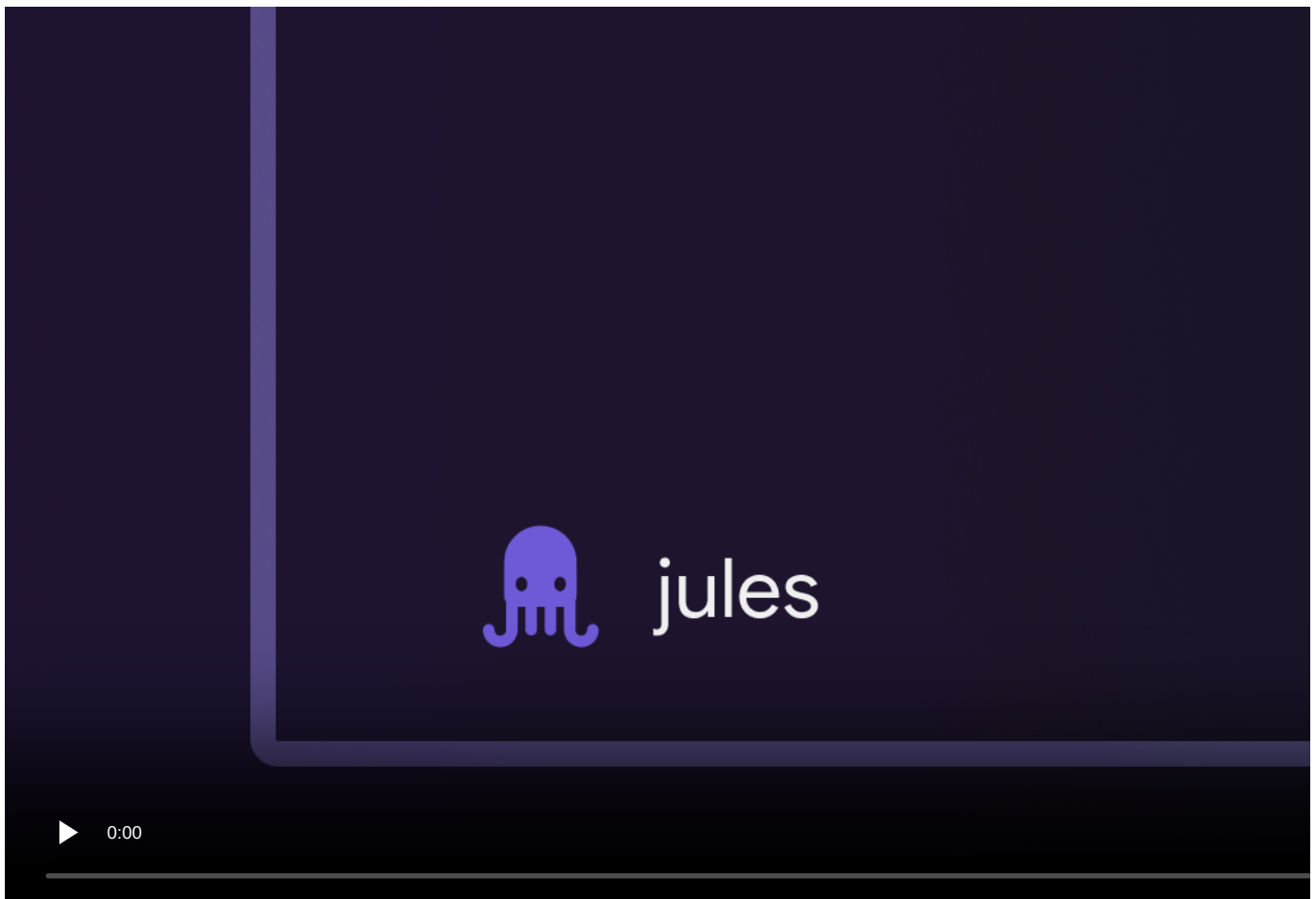
Continuous AI Overview

Módulo 4 | Fuente: <https://jules.google/docs/guides/continuous-ai-overview>

Continuous AI Overview

Launched

Continuous AI



Last week, we shipped **Continuous AI** at Jules. This is a massive shift in how you're going to work with Jules moving forward.

We've added three key features that turn Jules from a tool you use into a system that works for you. Here's the breakdown:

Suggested Tasks

Stop hunting through your codebase for that one `// TODO` you forgot about.

When you toggle the **Suggested Task** switch, Jules analyzes your entire codebase to find those floating comments and surfaces them as actionable tasks right in your dashboard.

Jules finds these tasks and analyzes them in context, providing a rationale for the change, and even giving you a confidence score on making that change independently. You can kick off multiple tasks in parallel and watch your to-do list shrink in real-time.

Pro-Tip: If you enable this and don't see any suggestions, it just means you're too organized! To get things moving, write a prompt asking Jules to analyze and raise specific inline to-dos. Once that's committed, Jules will surface those new tasks automatically.

Scheduled Tasks

This might be my personal favorite. **Scheduled Tasks** allow you to write a prompt and set it to run on a daily, weekly, or monthly interval.

This unlocks some incredible use cases for CI/CD. We've all been there: a build breaks, and you have to drop everything to fix it. Now, you can run those builds on a schedule, and if something breaks, Jules can step in and fix it for you.

In the video you'll see that I scheduled a task that creates a nightly release to NPM for a library. In the past, when it's been broken, Jules caught it, figured out the fix, and published it under the nightly tag.

Render Integration

To close the loop on Continuous AI, you need a tight feedback loop. That's why we've integrated Jules with **Render**.

Render is a fantastic cloud platform for hosting web services, and it has a great integration with GitHub that creates preview deployments for PRs. But sometimes, those builds fail.

Now, when a build fails on Render, it automatically reports that failure back to Jules. Jules then analyzes the error, works on a fix, and sends a commit back to your PR—which triggers a fresh, successful redeploy. It's an automated loop that keeps your deployment pipeline moving without manual intervention.

The Big Picture

When you combine **Suggested Tasks**, **Scheduled Tasks**, and the **Render Integration**, you aren't just using an AI assistant anymore. You're building a continuous system for your apps, websites, and libraries.

We are incredibly excited to see how you use these new features to keep your code moving.

Integrations

Módulo 4 | Fuente: <https://jules.google/docs/integrations/>

Integrations

Jules works best when it has the same context you do. By connecting your external tools, you enable Jules to autonomously detect bugs, read build logs, and understand project requirements without manual input.

Deployment & CI/CD

[Section titled "Deployment & CI/CD"](#)

Connect your deployment pipelines to let Jules automatically detect build failures and propose fixes.

[Render](#) Automatically retrieve build logs and fix deployment failures. Jules watches for failed builds, analyzes the logs, and pushes fixes directly to Jules' PRs.

How Integrations Work

[Section titled "How Integrations Work"](#)

Jules uses **scoped access** to interact with your tools. When you connect an integration:

1. **Read-Only by default:** Unless otherwise specified, Jules requests the minimum permissions needed to read logs or status updates.
2. **Autonomous triggers:** Integrations allow Jules to wake up based on external events (like a failed web hook) rather than waiting for you to type a prompt.
3. **Secure storage:** API keys are encrypted and stored securely. They are never exposed in the chat interface or shared between sessions.

Render Integration

Módulo 4 | Fuente: <https://jules.google/docs/integrations/render>

Render Integration

Turn deployment failures into instant fixes. By connecting Jules to Render, you empower it to watch your builds, analyze errors, and push fixes before you even switch context.

Prerequisites

[Section titled "Prerequisites"](#)

Before generating your API key, ensure your Render and GitHub environments are ready:

1. **Enable Pull Request Previews:** Go to your Render dashboard and [enable Pull Request Previews](#).
2. **Verify GitHub Permissions:** Check your [GitHub Installations](#).
 - Locate **Google Labs Jules**.
 - If you see a **Review Request** link next to it, click it to accept the updated permissions.
 - If there is no link, your permissions are already up to date.

Setup

[Section titled "Setup"](#)

1. **Provision a Key** Navigate to the [Render Dashboard](#). Click the **Help** menu (top-right) and select **Coding Agents** to open the provisioning screen.

Alternatively, visit dashboard.render.com/jules directly.

2. **Create the Key** Click **Create API key**. Copy the key displayed.

This key is shown only once. If you lose it, you will need to provision a new one.

3. **Connect to Jules** In Jules, go to [Settings > Integrations](#). Paste the key into the Render field and submit.

Automated Fixes

[Section titled "Automated Fixes"](#)

Once connected, Jules automatically monitors the Pull Requests **it creates** for build failures.

1. **Jules creates a PR:** When you accept a plan, Jules opens a Pull Request with the changes.
2. **Detection:** If the Render build for that PR fails, Jules automatically analyzes the logs.
3. **Resolution:** Jules writes a fix and pushes a new commit to the same branch.
4. **Merge:** You review the fix just like any other change and merge when ready.

Jules currently only monitors and fixes build failures on Pull Requests created by Jules. It does not automatically debug failures on PRs created by you or other team members.

Módulo 5: Registro de Cambios (Changelog)

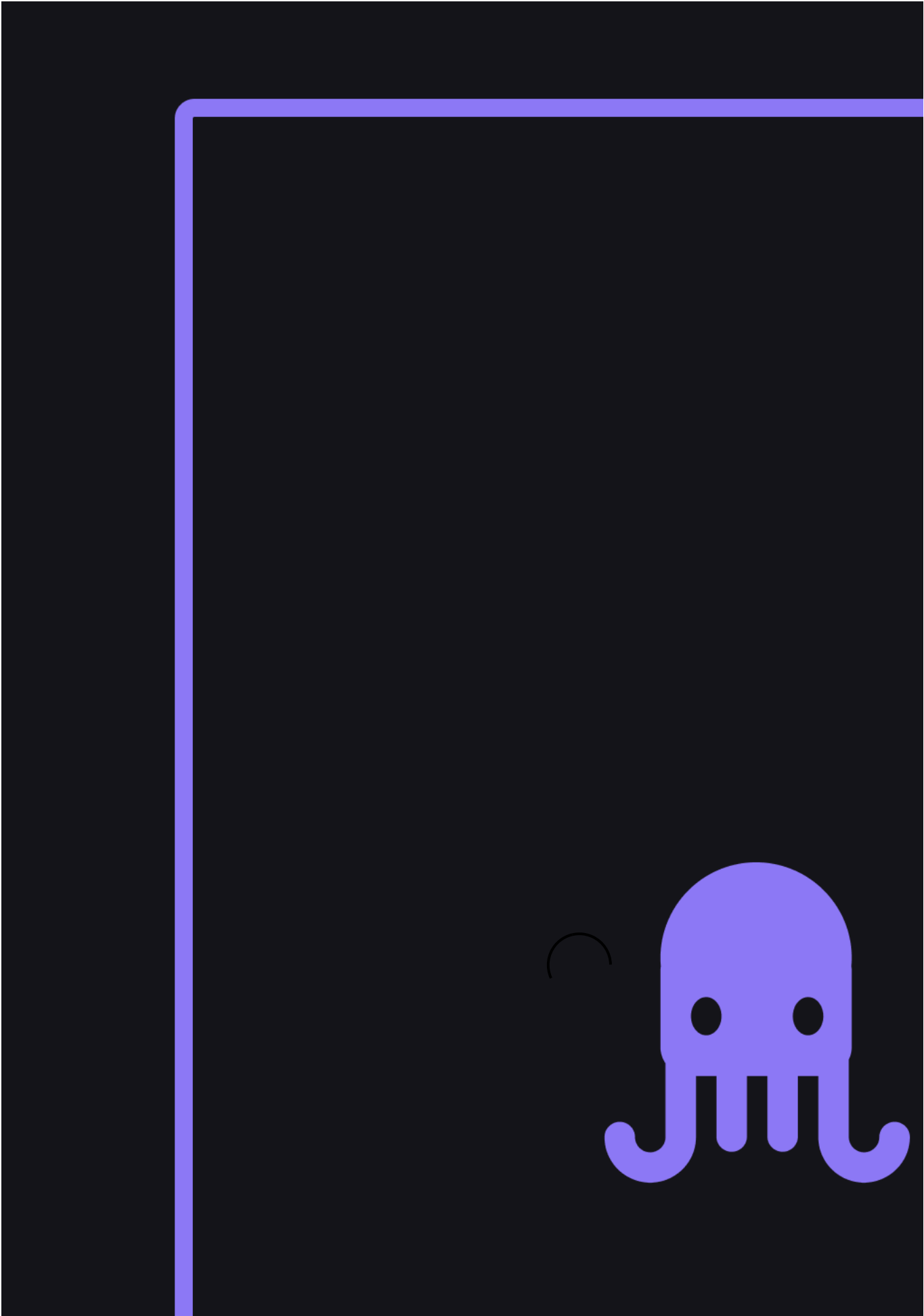
Changelog

Módulo 5 | Fuente: <https://jules.google/docs/changelog/>

Changelog

Gemini 3.1 Pro is now available in Jules for Google Pro plan users. 3.1 Pro is a smarter, more capable baseline for complex problem-solving.

This replaces Gemini 3 Pro as the default model for Pro users. Give it a try and let us know what you think.





CI Fixer

Jules now automatically detects and fixes CI failures on pull requests it creates. When a check fails in GitHub Actions, Jules receives the error, works through a fix, and resubmits the PR — all without manual intervention.

Previously, a failed CI check on a Jules PR would stall your workflow. Resolving it required setting up REST API calls or manually copy-pasting error logs back to Jules. CI Fixer closes that gap. Jules now operates in a loop — fixing, committing, and resubmitting — so your PRs keep moving forward.

Commit Authoring

You can now control how commits from Jules are attributed on GitHub. Three authorship modes are available:

- **Jules:** Jules is the sole author of all commits (default, existing behavior).
- **Co-authored (Jules + You or You + Jules):** Commits are co-authored by both you and Jules, crediting both contributors.
- **User only:** You are the sole author. Jules applies its changes under your identity.

Previously, Jules authored every commit and PR, which meant your GitHub contribution graph didn't reflect the work you initiated. Setting the commit authorship in to co-authored to you or you + Jules, makes it clear to contributors and PR reviewers who is driving each change.

It applies at the user level across all repositories and all task types — scheduled, suggested, and manually triggered tasks.

Setting commit authorship

1. Go to the Jules Settings page
2. Open the **Commit Authoring** section
3. Select your preferred authorship mode
4. All future sessions will use the selected attribution

Today we're launching MCP Server support in Jules starting with: Linear, Stitch, Neon, Tinybird, Context7, and Supabase.

Add a service connection in the Settings page and will trigger the MCP server tools in a session when it detects the need for a tool call.

Getting started

Our MCP server integration uses API key authentication. To get started:

1. Get the service's API key (such as Linear or Stitch)
2. Go to the Jules Settings page
3. Click on the MCP section
4. Plug in your API key for that specific service
5. Kick off a new session
6. Jules will trigger the MCP server tools when it detects the need for the call

Focus on Security

It might seem odd for us to limit which MCP servers Jules can connect to. However, we are wanted to start with a focus on security. Jules operates within a cloud development environment connected to your GitHub repositories, so we prioritize a **security-first** approach to third-party integrations.

By hand-selecting these initial servers, we can:

- **Validate Data Flow:** We ensure that each server adheres to strict data-handling standards, keeping your source code and API keys isolated.
- **Audit Tool Permissions:** We verify that each server only requests the specific permissions it needs to function, preventing "over-privileged" access to your environment.
- **Ensure Stability:** A vetted integration means fewer connection drops and a more predictable experience when Jules interacts with your external data.

We plan to expand this list as we work with more partners.

Tell us what to add next

If you have an MCP server in mind, you can let us know in the settings panel what we should add next.

Today we're launching Gemini 3 Flash in Jules for all users on all tiers.

It's our new base model that's faster and significantly more capable than our previous base model, Gemini 2.5 Pro. If it feels like Jules feels like it had a double espresso, you now know why.

Give it a spin and let us know what you think.

We've introduced a secondary agent—the Planning Critic—to review all plans that do not require human intervention. Before Jules executes a single line of code, this agent rigorously critiques and refines the proposed plan. While this adds a small amount of time to the planning phase, the result is a 9.5% reduction in task failure rates and significantly higher quality execution paths.

Jules now surfaces Performance Optimizations Jules has expanded its detection capabilities.

In addition to standard TODOs, Jules now proactively identifies performance optimizations within your code. These suggestions appear alongside your existing TODOs in the Suggested Tasks view, helping you catch bottlenecks and inefficiencies just as easily as you catch unfinished code.



```
PROMPT_CONTENT=$(jq -Rs . < my_prompt.md)

curl 'https://jules.googleapis.com/v1alpha/sessions' \
-X POST \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $JULES_API_KEY" \
-d "{
  \"prompt\": $PROMPT_CONTENT,
  \"
```

Repoless support in the REST API

Repoless comes to the REST API and we think this has **massive potential**. A repoless session in a sense becomes serverless, because it creates an ephemeral cloud dev environment with Node, Python, Rust, Bun, and other runtimes preloaded on the image.

The massive potential is that this serverless environment comes with an AI coding agent.

With a single API call you can spawn a serverless dev environment with only the context in your prompt. When it's done you can download the file outputs from the session.

We can't wait to see what you all do with this one.

Get the entire file outputs from a session

```
FILES CHANGED
-----
[M] bun.lock                                +567    -163
[A] bunfig.toml                             +2       -0
[M] examples/simple/main.ts                 +2       -1
[M] packages/core/src/client.ts              +21      -6
[M] packages/core/src/network/adapters.ts    +46     -13
[A] packages/core/src/retry-utils.ts         +89       -0
[M] packages/core/src/sources.ts             +2       -0
[M] packages/core/src/streaming.ts           +15     -51
[M] packages/core/src/types.ts              +12       -0
[M] packages/core/tests/automated_session.test.ts +125     -0
[M] packages/core/tests/network/adapters.test.ts +109     -0
[A] packages/core/tests/retry-utils.test.ts  +140     -0
[M] packages/core/tests/sources.test.ts      +21       -0

Summary: 13 files changed                  +1,151 insertions, -234

Session finished: completed
```

Speaking of file outputs, we just made that a lot easier. When a session completes a task, you can now get the entire change set.

This is stored in a git patch format for parsing changes such as additions, modifications, and deletions.

Timestamp based filters for Session Activities

```
ENDPOINT="https://jules.googleapis.com/v1alpha/sessions/1234567/activities"
TIMESTAMP="2026-01-17T00:03:53.137240Z"

curl -H "x-goog-api-key: $JULES_API_KEY" \
     "$ENDPOINT?createTime=$TIMESTAMP"
```

Each Jules Session has a collection of activities that represent the steps Jules took to complete the task. These activities are immutable and can be used to reconstruct the session.

Activities follow the event source pattern, meaning once they occur they never change. They are a record in time of the steps Jules took to complete the task. Since they never change, you can cache them aggressively.

We added a `createTime` filter that works like a range cursor and allows you to only fetch new activities over the network.

This may seem like a small optimization but stay tuned. We have some big news around this one.

Quick quality-of-life update: You no longer need to delete and recreate a scheduled task just to fix a typo or change the timing.

You can now **Edit**, **Pause**, and **Resume** your tasks directly from the menu. It's a small change that saves a lot of clicking.

Jules can now work proactively in the background to improve your code. With Suggested Tasks, Jules scans your repository for potential improvements and presents them on your repo page.

To get started, enable the proactive suggestions toggle on the repo page.

- **Tackle the #TODOs:** In this initial release, Jules focuses on identifying `#TODO` comments in your code. It reads the context, formulates a plan, and presents it for your approval—turning idle comments into active solutions. More use cases coming soon.
- **Continuous Improvement:** Once enabled, Jules continuously monitors your codebase. You don't have to ask; just check your dashboard for new suggestions and approve the ones you want.

This experimental feature is available today for Google AI Pro and Ultra subscribers on up to five repositories.

Read more at: [Suggested Tasks](#).

You can now set recurring tasks for Jules. Whether it's a weekly dependency check, a nightly lint fix, or a monthly cleanup, just define it once and Jules will handle the rest on your schedule.

To get started, navigate to the **Scheduled** tab on your repo page to configure your first task.

- **Set it and forget it:** Define your maintenance chores once. Jules will wake up, perform the task, and open a PR without you needing to lift a finger.
- **Never miss a beat:** No more manual prompts for the stuff that needs to happen every week. Ensure consistent code quality and dependency hygiene by automating your routine work.

This feature is available today for all Jules users.

Read more at: [Scheduled Tasks](#).

We've integrated with Render to handle the last mile of shipping code. Jules can now detect failed builds, analyze the logs, and push fixes to its PRs before you even know your build failed.

To enable this, go to your Render Dashboard, open the Help menu (top-right), and click Coding Agents to provision your API Key. Paste this key into Settings > Integrations in Jules.

1. **Instant Recovery:** No more context switching to dig through console logs. Jules detects the failure immediately and identifies the root cause.
2. **Proactive Fixes:** Instead of just alerting you to an error, Jules writes the code to fix it. Review the solution as a standard commit on your PR, merge it, and get back to green.

This integration is available now. Check out the [docs](#) for the full setup guide, including enabling PR previews.

Note: For now, Jules only fixes PRs it has created.

You can now start a Jules task immediately without selecting a repository. We've removed the speed bump between your idea and your code, allowing you to capture that spark of inspiration without the overhead.

To trigger this, simply click the "X" next to the selected repo to start a fresh, repoless session.

- **Skip the Detour:** Previously, starting a fresh journey meant hopping over to GitHub to create an empty repo first. Now, you can bypass that context switch entirely and keep your momentum.
- **Instant Ideation:** Whether you are prototyping a new feature or writing a quick script, you can dive straight into the logic. Just describe what you want, and Jules gets to work.

This update is available now for all users starting a new task.

Gemini 3 Pro is now available in Jules. This is the newest generation of the Gemini family, bringing clearer reasoning, stronger instruction following, and a meaningful lift in day-to-day reliability.

- **Coherent Planning:** Multi-step tasks hold together more naturally. The agent requires less management during transitions, meaning your work moves forward with fewer detours.
- **Visual Verification:** Leveraging the improved multimodal capabilities of Gemini 3 Pro, Jules renders and verifies web app outcomes with significantly higher precision.
- **Agentic Memories:** The new model utilizes context more effectively, helping Jules adapt to your coding preferences and project nuances more reliably over time.

Gemini 3 Pro is rolling out now to Google AI Ultra users and will be available to Pro users in the coming days.

We've been busy shipping a bunch of new updates to the Jules Tools CLI to make your experience smoother and more powerful. Here's a rundown of what's new:

Parallel Task Execution

You can now start multiple parallel tasks for the same prompt using the `--parallel` flag with `jules remote new`. This is useful for getting multiple suggestions from Jules at once (max of 5).

- **Added:** `--parallel` flag to `remote new` command.

v0.1.40 - WSL/Arch Linux Credential Fixes

We've refactored how we handle authentication to resolve credential issues for users on WSL and Arch Linux. This means broader platform support and no more login issues.

v0.1.39 - OAuth2 Error Handling Improvements

We've enhanced our OAuth2 flow to be more resilient with better error recovery, making the authentication process more reliable.

v0.1.38 - Repository Inference Feature

To shorten CLI commands and reduce configuration, we've added repository inference. Now, Jules can automatically detect the repository from your current directory, so you don't have to specify it manually.

v0.1.37 - PNPM Installation Fixes

We've added better support for the PNPM package manager, ensuring full compatibility for a wider range of JavaScript projects.

v0.1.36 - Side-by-Side Diff Viewer + Bug Fixes

Reviewing code is now faster and more readable with the new side-by-side diff viewer in the TUI. We've also added comprehensive test coverage and fixed bugs related to auto-approval and timeout validation.

You can now programmatically access Jules's capabilities to automate your work and build powerful integrations. The Jules API is designed to help you seamlessly integrate Jules into your existing development workflows, unlocking new ways to automate and enhance the entire software development lifecycle.

With the API, you can:

- Create custom integrations with tools like Slack for "ChatOps" workflows, allowing you to assign tasks directly from your chat client.
- Automate bug fixing and feature implementation by connecting Jules to your project management tools like Linear or Jira.
- Integrate Jules directly into your CI/CD pipelines in services like GitHub Actions.

Here's a quick example of how to create a new task (a "Session") using a cURL command:

```
curl 'https://jules.googleapis.com/v1alpha/sessions' \
-X POST \
-H "Content-Type: application/json" \
-H 'X-Goog-API-Key: YOUR_API_KEY' \
-d '{
  "prompt": "Create a boba app!",
  "sourceContext": {
    "source": "sources/github/bobalover/boba",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  },
  "title": "Boba App"
}'
```

For more examples see the [API documentation](#).

We're launching Jules Tools, a new command-line interface designed to give you direct control over your AI coding agent, making it scriptable, customizable, and easy to integrate into your existing workflows.

Key Features:

- **Direct Control:** Create tasks (`jules remote new`), list active sessions (`jules remote list`), and monitor Jules without leaving your command line.
- **Apply Patches Locally:** Instantly pull work-in-progress code from an active Jules session and apply it to your local machine. This lets you test changes immediately, without waiting for a commit to GitHub.
- **Scriptable & Composable:** Integrate Jules into your automations by piping in output from other tools like `gh`, `jq`, or `cat`.
- **Interactive Dashboard:** For a more guided experience, launch the built-in terminal user interface (TUI) to create and manage tasks step-by-step.

How to Install:

Install globally via npm: `npm install -g @google/jules`

Or run directly without a permanent installation: `npx @google/jules`

Starter Commands to Try:

See all available commands: `jules help`

List all repos connected to Jules: `jules remote list --repo`

Create a new task in a specific repo: `jules remote new --repo torvalds/linux --session "write unit tests"`

If you run into any issues, please share your experience with us via in-app feedback or on our [Discord channel](#).

You can now provide Jules with environment variables at the repository level. This enables Jules to access the project-specific configurations it needs to complete tasks, like running builds, executing tests, or interacting with different services.

How It Works:

- **Add Variables in Repo Settings:** Navigate to your repository's settings page to add your environment variables. They will be associated directly with your project.
- **Enable for a Task:** When you start a new task with Jules, you'll have the option to make these environment variables available to it.
- **Task-Long Access:** Once enabled for a specific task, Jules will have access to the variables for the entire duration of that task. Please note that this setting cannot be changed after the task has begun.

We're excited to see how this unlocks new and more complex workflows for you and your team! Let us know if you have any feedback.

Jules Memory for Repositories: We're excited to introduce a new Memory feature! Jules now has the ability to learn from your interactions.

- **How it works:** During a task, Jules will save your preferences, nudges, and corrections.
- **The benefit:** The next time you run the same or a similar task in that specific repository, Jules will reference its memory to better anticipate your needs and follow your established patterns, leading to more accurate results with less guidance.
- **Settings:** You can toggle memory on or off for the repo in the repo settings page under "Knowledge"

You can now tell Jules exactly which files to work with for any given task. Use the new file selector to easily and precisely reference specific files.

This removes ambiguity and gives you more granular control over Jules's actions, helping to tighten the context for your task.

Jules is now able to read and respond to your comments on pull requests!

When you start a review, Jules will add a 📖 emoji to each comment to let you know it's been read. Based on your feedback, Jules will then push a commit with the requested changes.

For more control, you can switch to **Reactive Mode** in your [global Jules UI settings](#). In this mode, Jules will only act on comments where you specifically mention `@Jules`.

Ahoy, mateys! To celebrate International Talk Like a Pirate Day, we've given Jules a temporary map to the treasure.

- Jules Speaks Pirate: You'll find your AI agent's responses are a bit more... swashbuckling... for today only.
- Same Great Logic: Fear not! Beneath the eyepatch and Jolly Roger, it's the same powerful coding engine ready to help you plunder that backlog and send bugs to Davy Jones' locker.

You can now upload images when creating a task in Jules. Use this to show frontend bugs, design inspiration, UI mocks, or any visual context you want Jules to consider while generating code.

For now:

- Only JPEG and PNG formats are supported.
- You can upload as many images as you want, as long as the total size is under 5MB.

- Image upload is only supported at task creation (we're working on enabling it for follow-up prompts soon).

Note: If your task involves using assets (e.g. logos) directly in code, those must still be committed to your GitHub repo.

[Read more](#) about Jules image support.

To improve the code review experience, we've introduced a new stacked layout for the diff viewer. This change displays diffs for multiple files vertically on a single screen. The stacked view makes it easier to see related changes across your codebase at a glance, providing better context and speeding up your review process.

Changes:

- The diff viewer now stacks file changes vertically by default
- You can also toggle back to the previous tabbed diff viewer

We've shipped significant improvements to the Jules critic agent, making its feedback more insightful and reliable. To increase transparency and give you more insight into its evaluation process, you can now see the critic's real-time analysis as it works.

Changes:

- The critic's thought process is now visible in the UI, showing its step-by-step evaluation of the code in real-time.
- The critic's now incorporates more contextual information when making decisions, leading to more accurate and relevant feedback on potential bugs and logic flaws.

To help new users get started with Jules, we've added sample prompts to the home page. These static prompts provide examples of how to use Jules and can be added to the text box with a single click.

Changes:

- Sample prompts are now displayed on the home page for all users.
- Clicking on a sample prompt will add the text of the prompt to the input box.

Jules now intelligently renders images within the diff viewer, providing an immediate visual context for your modifications.

This means:

- **Instant Visual Feedback:** When Jules generates images (like charts, diagrams, or web UI screenshots), you'll see the actual image in the diff, not just its code representation.
- **Streamlined Workflow:** No need to switch between tools or download files to see the results. Jules keeps everything in one place.

Try it out! Ask Jules to render an output, like a graph based on data, and commit it to your repository. You'll be able to see the generated image seamlessly within your diff viewer.

You're now in full control of when your code gets to GitHub. No need to wait for a task to finish or ask Jules to do it for you. At any point during a task, just click the GitHub icon in the top right to publish the current work-in-progress as a new branch or open a pull request. This gives you more flexibility and control to review, test, or take over whenever you're ready.

We heard your feedback about running into disk space limits on larger projects. To address this, we've significantly increased the available disk space in the Jules VM to 20GB. This provides more room for large dependencies, build artifacts, and complex repositories, reducing disk-related failures so Jules can tackle bigger tasks. Happy Julesing!

Jules can now proactively search the web for relevant content, documentation, or code snippets to help complete your tasks. This means Jules can get the information it needs, resulting in more accurate and successful task completion.

In Summary:

- Jules can find the latest documentation for dependencies/libraries you're using
- Jules can proactively find examples or code snippets that can help inform its implementation

Note: web search works best when working on technical documentation. Queries like: "What is the latest news today?" are not supported.

Meet Interactive Plan. Instead of jumping straight to the solution, Jules will now read your codebase, ask clarifying questions, and work with you to refine the plan. This collaborative approach gives you more control and ensures you're on the same page, leading to higher-quality code and a more reliable solution.

In summary:

- Trigger the interactive plan from the dropdown when you start a task
- Jules will start a brainstorm with you and ask clarifying questions

Great developers don't just write code, they question it. And now, so does Jules. We've built the Jules critic agent to ensure that every line of code isn't just functional, but robust, secure, and efficient. It acts as an internal peer reviewer, challenging every proposed change to elevate the quality of the final output.

Some high level notes:

- **Critic-augmented generation:** The Jules critic is integrated directly into the generation process. Every proposed change undergoes adversarial review before completion.
- **Improved code quality:** The critic flags subtle bugs, missed edge cases, and inefficient code. Jules then uses this feedback to improve the patch in real-time.
- **A new kind of review:** The critic is not just another linter or test. It understands the intent and context behind code, similar to a human peer reviewer.
- **Built on research:** This feature draws on research into multi-step, tool interactive critiquing and actor-critic reinforcement learning, where an "actor" generates and a "critic" evaluates.

Next time you are working on a front end project with Jules, ask it to verify its work and it'll render the website and send you back a screenshot!

- Ask Jules to complete a web development task and to verify the front end
- Jules will send you a screenshot of the front end along with any code changes
- The default Jules base image now includes Playwright for front end testing
- Users can also add images in the form of public URLs for Jules to use as input

Today we are thrilled to announce that Jules is no longer in beta! Since launch just two months ago, Jules has passed over 140k public commits. Thank you to our amazing beta users for all your support and feedback.

In addition, we're launching our pricing plans to unlock higher task limits, along with a bunch of quality improvements in the Jules app and agent. Here are the details:

- Get higher task limits through the Google AI Pro and Ultra plans. More details at [Limits and Plans](#).

- Jules now uses the power of Gemini 2.5 thinking when creating its plan, resulting in higher quality plans and more complete tasks
- Numerous bug fixes so Jules gets stuck less, and is better at following your instructions in agents.md

Jules now creates a snapshot of your environment when you add environment setup scripts. For complicated environment, users should see faster and more consistent task execution.

In summary:

- Jules will now snapshot your environment when you provide an environment setup script
- Snapshots are loaded automatically next time you run a task
- This provides for faster task startups, especially for complex environments
- You can find environment configuration by clicking the "codebase" in the left hand panel, or by clicking the "configure environment" button in the task pane.

Closing the loop from task to merge 🍷

Jules can now open a pull request directly from the UI. After a task completes, just use the new dropdown next to the 'Publish Branch' button to open a PR. Jules will request to merge the newly published branch into main, streamlining your entire workflow. Less context switching, faster merging.

Jules now supports [Bun](#). You can run tasks using Bun out of the box, no extra setup required. This expands compatibility for projects that use Bun instead of Node.

[Read more](#) about the jules base image and what tooling works with Jules.

- Pause, resume, and delete tasks—without losing your sense of place. Available from sidebar and repo view. You can even quickly copy task urls!
- Non-urgent task icons are now more recessive
- Certain hover states—which did not look good—have been toned back.
- System messages have more consistent padding and borders

[Learn more about running a task.](#)

Add the label 'jules' to any GitHub issue to start a task in Jules. That's it—label on, task live.

How to summon Jules:

- Open a GitHub issue.
- Click the gear next to "Labels".
- Add the label 'jules.'

Make sure the Jules GitHub App has access to your repo. After that, Jules takes it from there. [Read more about running tasks in Jules!](#)

We've shipped a big upgrade to the Jules agent under the hood.

What's new:

- **Smarter context.** Jules reads from AGENTS.md if it's in your repo.
- **Improved performance.** Tasks now complete faster—no numbers to share just yet, but you'll feel it.
- **Significantly reduced punting.** We tightened the loop to keep Jules moving forward.
- **More reliable setup.** If you've added an environment setup script, Jules now runs it consistently.
- **Better test habits.** Jules is more likely to write and run tests on its own.

Check out the [Getting Started](#) guide to learn more about AGENTS.md support.

We've overhauled the Jules development environment to move beyond the default Ubuntu 24.04 LTS packages. This includes:

- Explicitly installing newer versions of key toolchains like Rust, Node, and Python, addressing long-standing version issues.
- Adding finer-grained control over installation steps via custom scripts instead of relying solely on apt.
- Introducing support for multiple runtimes, improved isolation, and version pinning to reduce drift and better match developer expectations.

These changes unblock several issues developers encountered with outdated dependencies and improve alignment with modern project requirements.

[Read about the Jules environment setup to learn more about what's pre-installed.](#)

Performance upgrades: Enjoy a smoother, faster Jules experience with recent under-the-hood improvements.

Quickly copy and download code: New copy and download buttons are now available in the code view pane, making it easier to grab your code directly from Jules.

Stay focused with task modals: Initiate multiple tasks seamlessly through a new modal option, allowing you to keep your context and workflow intact. [Learn more](#) about kicking off tasks.

Adjustable code panel: Customize your workspace by adjusting the width of the code panel to your preferred viewing experience.

[Check out the docs](#) to learn more about how to download code that Jules writes.

This week, our focus has been on improving reliability, fixing our GitHub integration, and scaling capacity.

Here's what's we shipped:

- Updated our limits to 60 tasks per day, 5 concurrent.
- We substantially improved the reliability of the GitHub sync. Export to GitHub should also be fixed on previously created tasks.
- We've decreased the number of failure cases by 2/3

Learn more [about usage limits](#).

We've been heads down improving stability and fixing bugs—big and small—to make Jules faster, smoother, and more reliable for you.

Here's what's fixed:

- Upgraded our queuing system and added more compute to reduce wait times during peak usage
- Publish Branch button is now part of the summary UI in the activity feed so it's easier to find
- Bug vixes for task status and mobile

[Learn more](#) about how to publish a branch on GitHub.

Today, we're launching [Jules](#), a new AI coding agent.

Jules helps you move faster by working autonomously on tasks in your GitHub repo. It can fix bugs, update dependencies, migrate code, and add new features.

Once you give Jules a task, it spins up a fresh dev environment in a VM, installs dependencies, writes tests, makes the changes, runs the tests, and opens a pull request. Jules shows its work as it makes progress, so you never have to guess what code it's writing, or what it's thinking.

What Jules can do today

- Fix bugs with test verified patches
- Handle version bumps and dependency upgrades
- Perform scoped code transformations
- Migrate code across languages or frameworks
- Ship isolated, scoped, features
- Open PRs with runnable code and test results

[Get started with the Jules documentation](#), and visit jules.google.com to run your first Jules task.

Gemini 3.1 Pro is now available in Jules

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2026-03-09>

Gemini 3.1 Pro is now available in Jules

March 9, 2026

Gemini 3.1 Pro is now available in Jules for Google Pro plan users. 3.1 Pro is a smarter, more capable baseline for complex problem-solving.

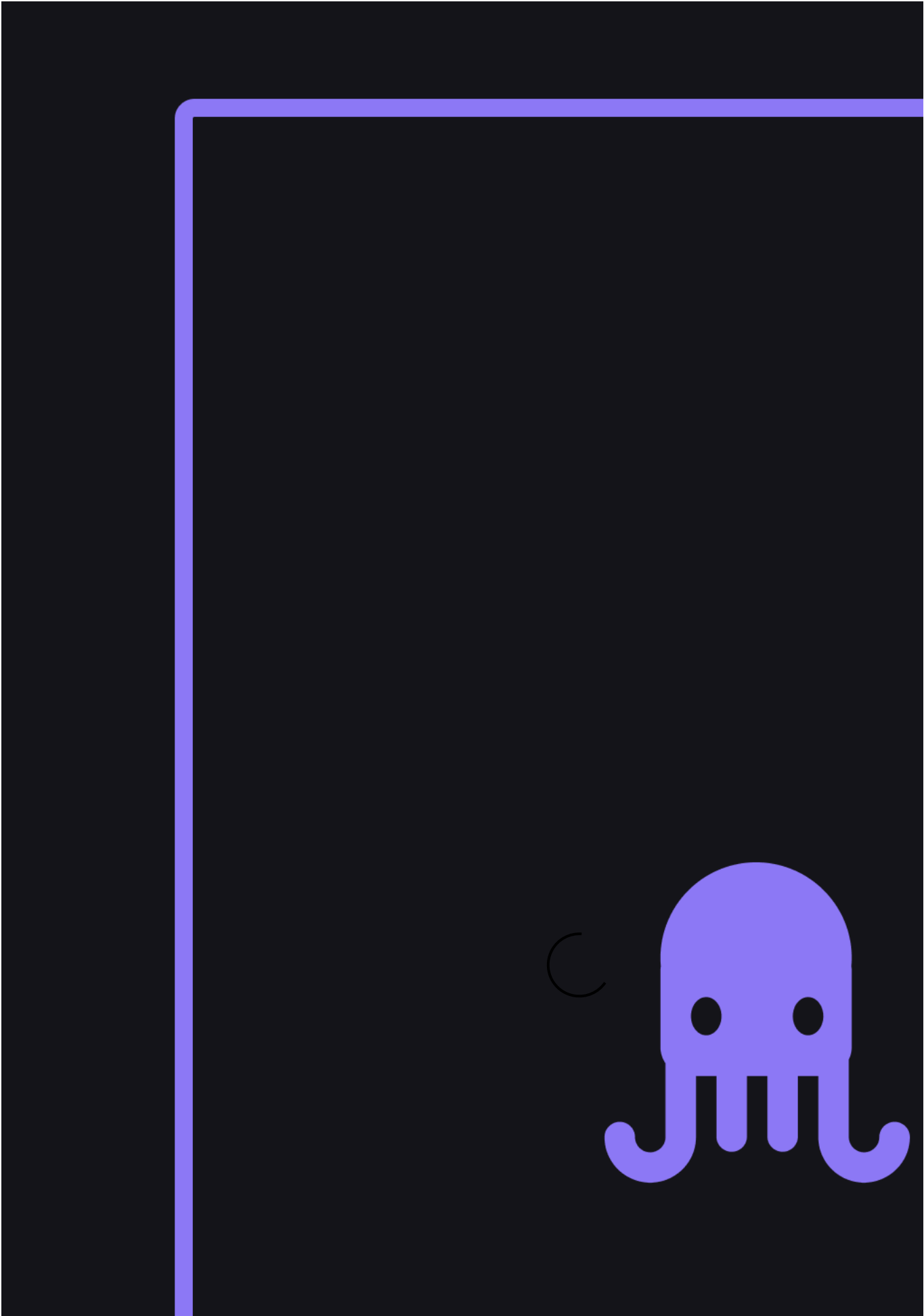
This replaces Gemini 3 Pro as the default model for Pro users. Give it a try and let us know what you think.

Auto-Fixing CI Failures and configure Jules to commit as you

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2026-02-19>

Auto-Fixing CI Failures and configure Jules to commit as you

February 19, 2026





CI Fixer

Jules now automatically detects and fixes CI failures on pull requests it creates. When a check fails in GitHub Actions, Jules receives the error, works through a fix, and resubmits the PR — all without manual intervention.

Previously, a failed CI check on a Jules PR would stall your workflow. Resolving it required setting up REST API calls or manually copy-pasting error logs back to Jules. CI Fixer closes that gap. Jules now operates in a loop — fixing, committing, and resubmitting — so your PRs keep moving forward.

Commit Authoring

You can now control how commits from Jules are attributed on GitHub. Three authorship modes are available:

- **Jules:** Jules is the sole author of all commits (default, existing behavior).
- **Co-authored (Jules + You or You + Jules):** Commits are co-authored by both you and Jules, crediting both contributors.
- **User only:** You are the sole author. Jules applies its changes under your identity.

Previously, Jules authored every commit and PR, which meant your GitHub contribution graph didn't reflect the work you initiated. Setting the commit authorship in to co-authored to you or you + Jules, makes it clear to contributors and PR reviewers who is driving each change.

It applies at the user level across all repositories and all task types — scheduled, suggested, and manually triggered tasks.

Setting commit authorship

1. Go to the Jules Settings page
2. Open the **Commit Authoring** section
3. Select your preferred authorship mode
4. All future sessions will use the selected attribution

MCP support comes to Jules

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2026-02-02>

MCP support comes to Jules

February 2, 2026

Today we're launching MCP Server support in Jules starting with: Linear, Stitch, Neon, Tinybird, Context7, and Supabase.

Add a service connection in the Settings page and will trigger the MCP server tools in a session when it detects the need for a tool call.

Getting started

Our MCP server integration uses API key authentication. To get started:

1. Get the service's API key (such as Linear or Stitch)
2. Go to the Jules Settings page
3. Click on the MCP section
4. Plug in your API key for that specific service
5. Kick off a new session
6. Jules will trigger the MCP server tools when it detects the need for the call

Focus on Security

It might seem odd for us to limit which MCP servers Jules can connect to. However, we wanted to start with a focus on security. Jules operates within a cloud development environment connected to your GitHub repositories, so we prioritize a **security-first** approach to third-party integrations.

By hand-selecting these initial servers, we can:

- **Validate Data Flow:** We ensure that each server adheres to strict data-handling standards, keeping your source code and API keys isolated.
- **Audit Tool Permissions:** We verify that each server only requests the specific permissions it needs to function, preventing "over-privileged" access to your environment.
- **Ensure Stability:** A vetted integration means fewer connection drops and a more predictable experience when Jules interacts with your external data.

We plan to expand this list as we work with more partners.

Tell us what to add next

If you have an MCP server in mind, you can let us know in the settings panel what we should add next.

Gemini 3 Flash is now the base model in Jules

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2026-01-30>

Gemini 3 Flash is now the base model in Jules

January 30, 2026

Today we're launching Gemini 3 Flash in Jules for all users on all tiers.

It's our new base model that's faster and significantly more capable than our previous base model, Gemini 2.5 Pro. If it feels like Jules feels like it had a double espresso, you now know why.

Give it a spin and let us know what you think.

Update Scheduled Tasks

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2026-01-26-5>

Update Scheduled Tasks

January 26, 2026

Quick quality-of-life update: You no longer need to delete and recreate a scheduled task just to fix a typo or change the timing.

You can now **Edit**, **Pause**, and **Resume** your tasks directly from the menu. It's a small change that saves a lot of clicking.

REST API: Repoless, File Outputs, and Activity Filters

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2026-01-26-4>

REST API: Repoless, File Outputs, and Activity Filters

January 26, 2026

```
PROMPT_CONTENT=$(jq -Rs . < my_prompt.md)

curl 'https://jules.googleapis.com/v1alpha/sessions' \
-X POST \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $JULES_API_KEY" \
-d "{
  \"prompt\": $PROMPT_CONTENT,
  \"
```

Repoless support in the REST API

Repoless comes to the REST API and we think this has **massive potential**. A repoless session in a sense becomes serverless, because it creates an ephemeral cloud dev environment with Node, Python, Rust, Bun, and other runtimes preloaded on the image.

The massive potential is that this serverless environment comes with an AI coding agent.

With a single API call you can spawn a serverless dev environment with only the context in your prompt. When it's done you can download the file outputs from the session.

We can't wait to see what you all do with this one.

Get the entire file outputs from a session

FILES CHANGED		
[M]	bun.lock	+567 -163
[A]	bunfig.toml	+2 -0
[M]	examples/simple/main.ts	+2 -1
[M]	packages/core/src/client.ts	+21 -6
[M]	packages/core/src/network/adapter.ts	+46 -13
[A]	packages/core/src/retry-utils.ts	+89 -0
[M]	packages/core/src/sources.ts	+2 -0
[M]	packages/core/src/streaming.ts	+15 -51
[M]	packages/core/src/types.ts	+12 -0
[M]	packages/core/tests/automated_session.test.ts	+125 -0
[M]	packages/core/tests/network/adapter.test.ts	+109 -0
[A]	packages/core/tests/retry-utils.test.ts	+140 -0
[M]	packages/core/tests/sources.test.ts	+21 -0
Summary: 13 files changed		+1,151 insertions, -234

Session finished: completed

Speaking of file outputs, we just made that a lot easier. When a session completes a task, you can now get the entire change set.

This is stored in a git patch format for parsing changes such as additions, modifications, and deletions.

Timestamp based filters for Session Activities

```
ENDPOINT="https://jules.googleapis.com/v1alpha/sessions/1234567/activities"
TIMESTAMP="2026-01-17T00:03:53.137240Z"

curl -H "x-goog-api-key: $JULES_API_KEY" \
  "$ENDPOINT?createTime=$TIMESTAMP"
```

Each Jules Session has a collection of activities that represent the steps Jules took to complete the task. These activities are immutable and can be used to reconstruct the session.

Activities follow the event source pattern, meaning once they occur they never change. They are a record in time of the steps Jules took to complete the task. Since they never change, you can cache them aggressively.

We added a `createTime` filter that works like a range cursor and allows you to only fetch new activities over the network.

This may seem like a small optimization but stay tuned. We have some big news around this one.

Jules now finds performance optimizations

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2026-01-26-3>

Jules now finds performance optimizations

January 26, 2026

Jules now surfaces Performance Optimizations Jules has expanded its detection capabilities.

In addition to standard TODOs, Jules now proactively identifies performance optimizations within your code. These suggestions appear alongside your existing TODOs in the Suggested Tasks view, helping you catch bottlenecks and inefficiencies just as easily as you catch unfinished code.

Introducing the Planning Critic for Auto-Approved Plans

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2026-01-26-1>

Introducing the Planning Critic for Auto-Approved Plans

January 26, 2026

We've introduced a secondary agent—the Planning Critic—to review all plans that do not require human intervention. Before Jules executes a single line of code, this agent rigorously critiques and refines the proposed plan. While this adds a small amount of time to the planning phase, the result is a 9.5% reduction in task failure rates and significantly higher quality execution paths.

Automated fixes for Render deployments

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-12-102>

Automated fixes for Render deployments

December 10, 2025

We've integrated with Render to handle the last mile of shipping code. Jules can now detect failed builds, analyze the logs, and push fixes to its PRs before you even know your build failed.

To enable this, go to your Render Dashboard, open the Help menu (top-right), and click Coding Agents to provision your API Key. Paste this key into Settings > Integrations in Jules.

1. **Instant Recovery:** No more context switching to dig through console logs. Jules detects the failure immediately and identifies the root cause.
2. **Proactive Fixes:** Instead of just alerting you to an error, Jules writes the code to fix it. Review the solution as a standard commit on your PR, merge it, and get back to green.

This integration is available now. Check out the [docs](#) for the full setup guide, including enabling PR previews.

Note: For now, Jules only fixes PRs it has created.

Put routine maintenance on autopilot with Scheduled Tasks

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-12-101>

Put routine maintenance on autopilot with Scheduled Tasks

December 10, 2025

You can now set recurring tasks for Jules. Whether it's a weekly dependency check, a nightly lint fix, or a monthly cleanup, just define it once and Jules will handle the rest on your schedule.

To get started, navigate to the **Scheduled** tab on your repo page to configure your first task.

- **Set it and forget it:** Define your maintenance chores once. Jules will wake up, perform the task, and open a PR without you needing to lift a finger.
- **Never miss a beat:** No more manual prompts for the stuff that needs to happen every week. Ensure consistent code quality and dependency hygiene by automating your routine work.

This feature is available today for all Jules users.

Read more at: [Scheduled Tasks](#).

Enable suggested tasks to let Jules find issues proactively

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-12-10>

Enable suggested tasks to let Jules find issues proactively

December 10, 2025

Jules can now work proactively in the background to improve your code. With Suggested Tasks, Jules scans your repository for potential improvements and presents them on your repo page.

To get started, enable the proactive suggestions toggle on the repo page.

- **Tackle the #TODOs:** In this initial release, Jules focuses on identifying `#TODO` comments in your code. It reads the context, formulates a plan, and presents it for your approval—turning idle comments into active solutions. More use cases coming soon.
- **Continuous Improvement:** Once enabled, Jules continuously monitors your codebase. You don't have to ask; just check your dashboard for new suggestions and approve the ones you want.

This experimental feature is available today for Google AI Pro and Ultra subscribers on up to five repositories.

Read more at: [Suggested Tasks](#).

Start from scratch—instantly

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-11-20>

Start from scratch—instantly

November 20, 2025

You can now start a Jules task immediately without selecting a repository. We've removed the speed bump between your idea and your code, allowing you to capture that spark of inspiration without the overhead.

To trigger this, simply click the "X" next to the selected repo to start a fresh, repoless session.

- **Skip the Detour:** Previously, starting a fresh journey meant hopping over to GitHub to create an empty repo first. Now, you can bypass that context switch entirely and keep your momentum.
- **Instant Ideation:** Whether you are prototyping a new feature or writing a quick script, you can dive straight into the logic. Just describe what you want, and Jules gets to work.

This update is available now for all users starting a new task.

Introducing Gemini 3 Pro

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-11-19>

Introducing Gemini 3 Pro

November 19, 2025

Gemini 3 Pro is now available in Jules. This is the newest generation of the Gemini family, bringing clearer reasoning, stronger instruction following, and a meaningful lift in day-to-day reliability.

- **Coherent Planning:** Multi-step tasks hold together more naturally. The agent requires less management during transitions, meaning your work moves forward with fewer detours.
- **Visual Verification:** Leveraging the improved multimodal capabilities of Gemini 3 Pro, Jules renders and verifies web app outcomes with significantly higher precision.
- **Agentic Memories:** The new model utilizes context more effectively, helping Jules adapt to your coding preferences and project nuances more reliably over time.

Gemini 3 Pro is rolling out now to Google AI Ultra users and will be available to Pro users in the coming days.

New Jules Tools CLI Updates: Side-by-Side Diffs, Repo Inference, and More

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-11-10>

New Jules Tools CLI Updates: Side-by-Side Diffs, Repo Inference, and More

November 10, 2025

We've been busy shipping a bunch of new updates to the Jules Tools CLI to make your experience smoother and more powerful. Here's a rundown of what's new:

Parallel Task Execution

You can now start multiple parallel tasks for the same prompt using the `--parallel` flag with `jules remote new`. This is useful for getting multiple suggestions from Jules at once (max of 5).

- **Added:** `--parallel` flag to `remote new` command.

v0.1.40 - WSL/Arch Linux Credential Fixes

We've refactored how we handle authentication to resolve credential issues for users on WSL and Arch Linux. This means broader platform support and no more login issues.

v0.1.39 - OAuth2 Error Handling Improvements

We've enhanced our OAuth2 flow to be more resilient with better error recovery, making the authentication process more reliable.

v0.1.38 - Repository Inference Feature

To shorten CLI commands and reduce configuration, we've added repository inference. Now, Jules can automatically detect the repository from your current directory, so you don't have to specify it manually.

v0.1.37 - PNPM Installation Fixes

We've added better support for the PNPM package manager, ensuring full compatibility for a wider range of JavaScript projects.

v0.1.36 - Side-by-Side Diff Viewer + Bug Fixes

Reviewing code is now faster and more readable with the new side-by-side diff viewer in the TUI. We've also added comprehensive test coverage and fixed bugs related to auto-approval and timeout validation.

Introducing the Jules API

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-10-03>

Introducing the Jules API

October 3, 2025

You can now programmatically access Jules's capabilities to automate your work and build powerful integrations. The Jules API is designed to help you seamlessly integrate Jules into your existing development workflows, unlocking new ways to automate and enhance the entire software development lifecycle.

With the API, you can:

- Create custom integrations with tools like Slack for "ChatOps" workflows, allowing you to assign tasks directly from your chat client.
- Automate bug fixing and feature implementation by connecting Jules to your project management tools like Linear or Jira.
- Integrate Jules directly into your CI/CD pipelines in services like GitHub Actions.

Here's a quick example of how to create a new task (a "Session") using a cURL command:

```
curl 'https://jules.googleapis.com/v1alpha/sessions' \  
-X POST \  
-H "Content-Type: application/json" \  
-H 'X-Goog-API-Key: YOUR_API_KEY' \  
-d '{  
  "prompt": "Create a boba app!",  
  "sourceContext": {  
    "source": "sources/github/bobalover/boba",  
    "githubRepoContext": {  
      "startingBranch": "main"  
    }  
  },  
  "title": "Boba App"  
'
```

For more examples see the [API documentation](#).

Jules in the command line

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-10-02>

Jules in the command line

October 2, 2025

We're launching Jules Tools, a new command-line interface designed to give you direct control over your AI coding agent, making it scriptable, customizable, and easy to integrate into your existing workflows.

Key Features:

- **Direct Control:** Create tasks (`jules remote new`), list active sessions (`jules remote list`), and monitor Jules without leaving your command line.
- **Apply Patches Locally:** Instantly pull work-in-progress code from an active Jules session and apply it to your local machine. This lets you test changes immediately, without waiting for a commit to GitHub.
- **Scriptable & Composable:** Integrate Jules into your automations by piping in output from other tools like `gh`, `jq`, or `cat`.
- **Interactive Dashboard:** For a more guided experience, launch the built-in terminal user interface (TUI) to create and manage tasks step-by-step.

How to Install:

Install globally via npm: `npm install -g @google/jules`

Or run directly without a permanent installation: `npx @google/jules`

Starter Commands to Try:

See all available commands: `jules help`

List all repos connected to Jules: `jules remote list --repo`

Create a new task in a specific repo: `jules remote new --repo torvalds/linux --session "write unit tests"`

If you run into any issues, please share your experience with us via in-app feedback or on our [Discord channel](#).

Use Environment Variables In Jules

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-10-01>

Use Environment Variables In Jules

October 1, 2025

You can now provide Jules with environment variables at the repository level. This enables Jules to access the project-specific configurations it needs to complete tasks, like running builds, executing tests, or interacting with different services.

How It Works:

- **Add Variables in Repo Settings:** Navigate to your repository's settings page to add your environment variables. They will be associated directly with your project.
- **Enable for a Task:** When you start a new task with Jules, you'll have the option to make these environment variables available to it.
- **Task-Long Access:** Once enabled for a specific task, Jules will have access to the variables for the entire duration of that task. Please note that this setting cannot be changed after the task has begun.

We're excited to see how this unlocks new and more complex workflows for you and your team! Let us know if you have any feedback.

Jules gains memory!

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-09-30>

Jules gains memory!

September 30, 2025

Jules Memory for Repositories: We're excited to introduce a new Memory feature! Jules now has the ability to learn from your interactions.

- **How it works:** During a task, Jules will save your preferences, nudges, and corrections.
- **The benefit:** The next time you run the same or a similar task in that specific repository, Jules will reference its memory to better anticipate your needs and follow your established patterns, leading to more accurate results with less guidance.
- **Settings:** You can toggle memory on or off for the repo in the repo settings page under "Knowledge"

Tell Jules exactly what file to work on using file selector

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-09-29>

Tell Jules exactly what file to work on using file selector

September 29, 2025

You can now tell Jules exactly which files to work with for any given task. Use the new file selector to easily and precisely reference specific files.

This removes ambiguity and gives you more granular control over Jules's actions, helping to tighten the context for your task.

Jules Acts on PR Feedback

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-09-23>

Jules Acts on PR Feedback

September 23, 2025

Jules is now able to read and respond to your comments on pull requests!

When you start a review, Jules will add a 🗨️ emoji to each comment to let you know it's been read. Based on your feedback, Jules will then push a commit with the requested changes.

For more control, you can switch to **Reactive Mode** in your [global Jules UI settings](#). In this mode, Jules will only act on comments where you specifically mention `@Jules`.

All Hands on Deck!

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-09-19>

All Hands on Deck!

September 19, 2025

Ahoy, mateys! To celebrate International Talk Like a Pirate Day, we've given Jules a temporary map to the treasure.

- Jules Speaks Pirate: You'll find your AI agent's responses are a bit more... swashbuckling... for today only.
- Same Great Logic: Fear not! Beneath the eyepatch and Jolly Roger, it's the same powerful coding engine ready to help you plunder that backlog and send bugs to Davy Jones' locker.

Image upload

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-09-09>

Image upload

September 9, 2025

You can now upload images when creating a task in Jules. Use this to show frontend bugs, design inspiration, UI mocks, or any visual context you want Jules to consider while generating code.

For now:

- Only JPEG and PNG formats are supported.
- You can upload as many images as you want, as long as the total size is under 5MB.
- Image upload is only supported at task creation (we're working on enabling it for follow-up prompts soon).

Note: If your task involves using assets (e.g. logos) directly in code, those must still be committed to your GitHub repo.

[Read more](#) about Jules image support.

Stacked Diff

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-09-04>

Stacked Diff

September 4, 2025

To improve the code review experience, we've introduced a new stacked layout for the diff viewer. This change displays diffs for multiple files vertically on a single screen. The stacked view makes it easier to see related changes across your codebase at a glance, providing better context and speeding up your review process.

Changes:

- The diff viewer now stacks file changes vertically by default
- You can also toggle back to the previous tabbed diff viewer

Improved Jules Critic

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-09-03>

Improved Jules Critic

September 3, 2025

We've shipped significant improvements to the Jules critic agent, making its feedback more insightful and reliable. To increase transparency and give you more insight into its evaluation process, you can now see the critic's real-time analysis as it works.

Changes:

- The critic's thought process is now visible in the UI, showing its step-by-step evaluation of the code in real-time.
- The critic's now incorporates more contextual information when making decisions, leading to more accurate and relevant feedback on potential bugs and logic flaws.

Jules Sample Prompts

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-09-02>

Jules Sample Prompts

September 2, 2025

To help new users get started with Jules, we've added sample prompts to the home page. These static prompts provide examples of how to use Jules and can be added to the text box with a single click.

Changes:

- Sample prompts are now displayed on the home page for all users.
- Clicking on a sample prompt will add the text of the prompt to the input box.

Render images in the diff viewer

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-22>

Render images in the diff viewer

August 22, 2025

Jules now intelligently renders images within the diff viewer, providing an immediate visual context for your modifications.

This means:

- **Instant Visual Feedback:** When Jules generates images (like charts, diagrams, or web UI screenshots), you'll see the actual image in the diff, not just its code representation.
- **Streamlined Workflow:** No need to switch between tools or download files to see the results. Jules keeps everything in one place.

Try it out! Ask Jules to render an output, like a graph based on data, and commit it to your repository. You'll be able to see the generated image seamlessly within your diff viewer.

Increasing the VM Size to 20GB

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-151>

Increasing the VM Size to 20GB

August 15, 2025

We heard your feedback about running into disk space limits on larger projects. To address this, we've significantly increased the available disk space in the Jules VM to 20GB. This provides more room for large dependencies, build artifacts, and complex repositories, reducing disk-related failures so Jules can tackle bigger tasks. Happy Julesing!

Export at any time

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-15>

Export at any time

August 15, 2025

You're now in full control of when your code gets to GitHub. No need to wait for a task to finish or ask Jules to do it for you. At any point during a task, just click the GitHub icon in the top right to publish the current work-in-progress as a new branch or open a pull request. This gives you more flexibility and control to review, test, or take over whenever you're ready.

Critic Agent

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-083>

Critic Agent

August 8, 2025

Great developers don't just write code, they question it. And now, so does Jules. We've built the Jules critic agent to ensure that every line of code isn't just functional, but robust, secure, and efficient. It acts as an internal peer reviewer, challenging every proposed change to elevate the quality of the final output.

Some high level notes:

- **Critic-augmented generation:** The Jules critic is integrated directly into the generation process. Every proposed change undergoes adversarial review before completion.
- **Improved code quality:** The critic flags subtle bugs, missed edge cases, and inefficient code. Jules then uses this feedback to improve the patch in real-time.
- **A new kind of review:** The critic is not just another linter or test. It understands the intent and context behind code, similar to a human peer reviewer.
- **Built on research:** This feature draws on research into multi-step, tool interactive critiquing and actor-critic reinforcement learning, where an "actor" generates and a "critic" evaluates.

Interactive Plan

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-082>

Interactive Plan

August 8, 2025

Meet Interactive Plan. Instead of jumping straight to the solution, Jules will now read your codebase, ask clarifying questions, and work with you to refine the plan. This collaborative approach gives you more control and ensures you're on the same page, leading to higher-quality code and a more reliable solution.

In summary:

- Trigger the interactive plan from the dropdown when you start a task
- Jules will start a brainstorm with you and ask clarifying questions

Jules can surf the web

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-081>

Jules can surf the web

August 8, 2025

Jules can now proactively search the web for relevant content, documentation, or code snippets to help complete your tasks. This means Jules can get the information it needs, resulting in more accurate and successful task completion.

In Summary:

- Jules can find the latest documentation for dependencies/libraries you're using
- Jules can proactively find examples or code snippets that can help inform its implementation

Note: web search works best when working on technical documentation. Queries like: "What is the latest news today?" are not supported.

Jules can test web-apps and show you the results

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-07>

Jules can test web-apps and show you the results

August 7, 2025

Next time you are working on a front end project with Jules, ask it to verify its work and it'll render the website and send you back a screenshot!

- Ask Jules to complete a web development task and to verify the front end
- Jules will send you a screenshot of the front end along with any code changes
- The default Jules base image now includes Playwright for front end testing
- Users can also add images in the form of public URLs for Jules to use as input

Jules is out of beta!

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-06>

Jules is out of beta!

August 6, 2025

Today we are thrilled to announce that Jules is no longer in beta! Since launch just two months ago, Jules has passed over 140k public commits. Thank you to our amazing beta users for all your support and feedback.

In addition, we're launching our pricing plans to unlock higher task limits, along with a bunch of quality improvements in the Jules app and agent. Here are the details:

- Get higher task limits through the Google AI Pro and Ultra plans. More details at [Limits and Plans](#).
- Jules now uses the power of Gemini 2.5 thinking when creating its plan, resulting in higher quality plans and more complete tasks
- Numerous bug fixes so Jules gets stuck less, and is better at following your instructions in agents.md

Environment snapshots for faster tasks

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-05>

Environment snapshots for faster tasks

August 5, 2025

Jules now creates a snapshot of your environment when you add environment setup scripts. For complicated environment, users should see faster and more consistent task execution.

In summary:

- Jules will now snapshot your environment when you provide an environment setup script
- Snapshots are loaded automatically next time you run a task
- This provides for faster task startups, especially for complex environments
- You can find environment configuration by clicking the "codebase" in the left hand panel, or by clicking the "configure environment" button in the task pane.

Open A PR directly from Jules

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-08-04>

Open A PR directly from Jules

August 4, 2025

Closing the loop from task to merge 🍷

Jules can now open a pull request directly from the UI. After a task completes, just use the new dropdown next to the 'Publish Branch' button to open a PR. Jules will request to merge the newly published branch into main, streamlining your entire workflow. Less context switching, faster merging.

Added Bun runtime support

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-07-18>

Added Bun runtime support

July 18, 2025

Jules now supports [Bun](#). You can run tasks using Bun out of the box, no extra setup required. This expands compatibility for projects that use Bun instead of Node.

[Read more](#) about the jules base image and what tooling works with Jules.

Improved task controls and other 🧰 UI delight

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-07-03>

Improved task controls and other 🧰 UI delight

July 3, 2025

- Pause, resume, and delete tasks—without losing your sense of place. Available from sidebar and repo view. You can even quickly copy task urls!
- Non-urgent task icons are now more recessive
- Certain hover states—which did not look good—have been toned back.
- System messages have more consistent padding and borders

[Learn more about running a task.](#)

Jules now listens to GitHub issues

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-06-26>

Jules now listens to GitHub issues

June 26, 2025

Add the label 'jules' to any GitHub issue to start a task in Jules. That's it—label on, task live.

How to summon Jules:

- Open a GitHub issue.
- Click the gear next to "Labels".
- Add the label 'jules.'

Make sure the Jules GitHub App has access to your repo. After that, Jules takes it from there. [Read more about running tasks in Jules!](#)

Jules Agent Update: Faster, Smarter, More Reliable

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-06-20>

Jules Agent Update: Faster, Smarter, More Reliable

June 20, 2025

We've shipped a big upgrade to the Jules agent under the hood.

What's new:

- **Smarter context.** Jules reads from AGENTS.md if it's in your repo.
- **Improved performance.** Tasks now complete faster—no numbers to share just yet, but you'll feel it.
- **Significantly reduced punting.** We tightened the loop to keep Jules moving forward.
- **More reliable setup.** If you've added an environment setup script, Jules now runs it consistently.
- **Better test habits.** Jules is more likely to write and run tests on its own.

Check out the [Getting Started](#) guide to learn more about AGENTS.md support.

Modernized base environment and updated toolchains

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-06-18>

Modernized base environment and updated toolchains

June 18, 2025

We've overhauled the Jules development environment to move beyond the default Ubuntu 24.04 LTS packages. This includes:

- Explicitly installing newer versions of key toolchains like Rust, Node, and Python, addressing long-standing version issues.
- Adding finer-grained control over installation steps via custom scripts instead of relying solely on apt.
- Introducing support for multiple runtimes, improved isolation, and version pinning to reduce drift and better match developer expectations.

These changes unblock several issues developers encountered with outdated dependencies and improve alignment with modern project requirements.

[Read about the Jules environment setup to learn more about what's pre-installed.](#)

Customization and Efficiency Enhancements

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-06-06>

Customization and Efficiency Enhancements

June 6, 2025

Performance upgrades: Enjoy a smoother, faster Jules experience with recent under-the-hood improvements.

Quickly copy and download code: New copy and download buttons are now available in the code view pane, making it easier to grab your code directly from Jules.

Stay focused with task modals: Initiate multiple tasks seamlessly through a new modal option, allowing you to keep your context and workflow intact. [Learn more](#) about kicking off tasks.

Adjustable code panel: Customize your workspace by adjusting the width of the code panel to your preferred viewing experience.

[Check out the docs](#) to learn more about how to download code that Jules writes.

A faster, smoother and more reliable Jules

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-05-30>

A faster, smoother and more reliable Jules

May 30, 2025

This week, our focus has been on improving reliability, fixing our GitHub integration, and scaling capacity.

Here's what's we shipped:

- Updated our limits to 60 tasks per day, 5 concurrent.
- We substantially improved the reliability of the GitHub sync. Export to GitHub should also be fixed on previously created tasks.
- We've decreased the number of failure cases by 2/3

Learn more [about usage limits](#).

Improving Stability

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-05-22>

Improving Stability

May 22, 2025

We've been heads down improving stability and fixing bugs—big and small—to make Jules faster, smoother, and more reliable for you.

Here's what's fixed:

- Upgraded our queuing system and added more compute to reduce wait times during peak usage
- Publish Branch button is now part of the summary UI in the activity feed so it's easier to find
- Bug vixes for task status and mobile

[Learn more](#) about how to publish a branch on GitHub.

Jules is here

Módulo 5 | Fuente: <https://jules.google/docs/changelog/2025-05-19>

Jules is here

May 19, 2025

Today, we're launching [Jules](#), a new AI coding agent.

Jules helps you move faster by working autonomously on tasks in your GitHub repo. It can fix bugs, update dependencies, migrate code, and add new features.

Once you give Jules a task, it spins up a fresh dev environment in a VM, installs dependencies, writes tests, makes the changes, runs the tests, and opens a pull request. Jules shows its work as it makes progress, so you never have to guess what code it's writing, or what it's thinking.

What Jules can do today

- Fix bugs with test verified patches
- Handle version bumps and dependency upgrades
- Perform scoped code transformations
- Migrate code across languages or frameworks
- Ship isolated, scoped, features
- Open PRs with runnable code and test results

[Get started with the Jules documentation](#), and visit jules.google.com to run your first Jules task.